

Que. What is Computer?

- Computer is an electronic device, which made up of hardware and software component.
- It is used to store the data, process the data and retrieve the data.

Que. What is Hardware?

- It is combination of electronic component.

Hardware's such as:

- 1.Processor
- 2.RAM
- 3.ROM
- 4.Graphic Card
- 5.Cooling Fans
- 6.SMPS
- 7.Drivers & Ports.

Que. What is Language?

- A language is medium of communication.

Que. What is Programming language?

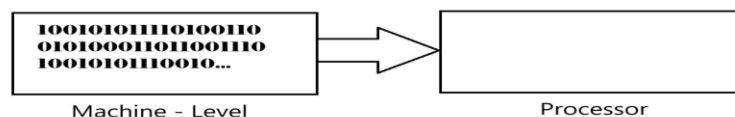
- A language which is used to command the computer to perform a specific task is known as programming language.

Levels of Programming Language:**3 Levels of programming language**

1. Low-Level Language
2. Mid-Level Language
3. High-Level Language

1. Low-Level Language:

- The Language which is directly understandable to computer or processor is known as Low-Level Language.
- Example: Binary Language.

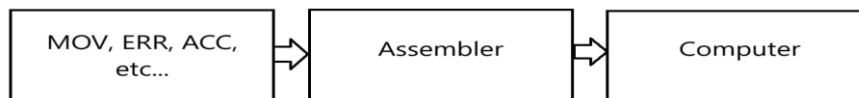


2. Mid-Level Language:

*The language which consisted of pre-defined set of code-words which is understandable to the computer by using a software is known as **Assembler**.*

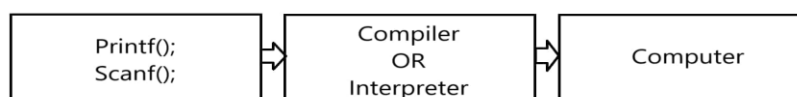
Q. What is Assembler?

An Intermediate software which is used to convert assembly level language into machine-level language.



3. High-Level Language:

- *The language which works very close to Human Languages & which is easily readable, understandable and instruct-able by a programmer to the computer is called as High-Level Language.*
- *The High-Level language is converted to machine-level language using an intermediate software known as compiler & interpreter.*
- Example: C, C++, Java, Python, Swift, R, Ruby



Software:

- *It is an interface between user and hardware.*

Platform:

- *Combination of software and hardware components on which application can run.*

Types of Software:

1. Platform Dependent
2. Platform Independent

1. Platform Dependent:

The software which are designed on a specific platform, if they can run on a same type of platforms but not on other platforms are known as platform dependent software's.

Example: C, C++

2. Platform Independent:

The software which are designed on a specific platform, which can run on any type of platforms are known as platform In-dependent software's.

Que. Why Software's made using C & C++ are platform Dependent?

- *If a program is written in C & C++ language and saved with an extension of ".c", then it is given to the C compiler. It converts ".c" file into ".exe" file.*
- *The ".exe" file contains **Native Codes** or Native **Machine-Level Codes***
- *The exe file can run only on the Native type of platforms. Example: Program made using windows platform can run only on windows type of platform but not on Mac-OS or Linux etc.*

Platform Independent:**Que. Why Software's made using Java language is Platform Independent?**

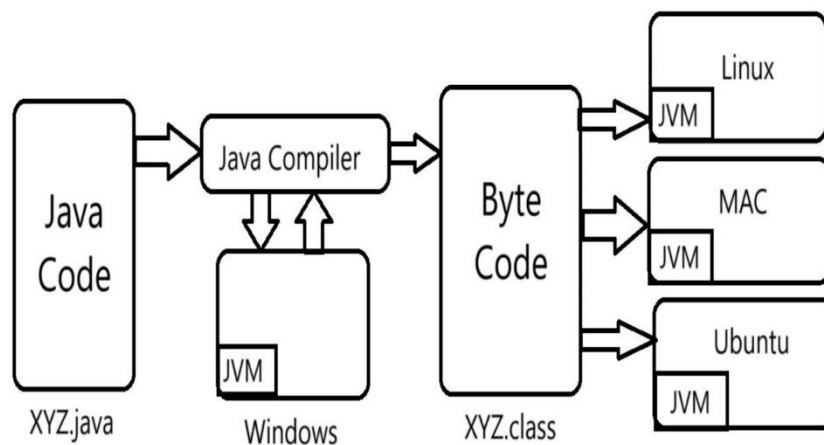
- *When a programmer writes a code using java language then save it in ".java" extension.*
- *When ".java" file is given to the java compiler, the java compiler converts ".java" file into ".class" file.*
- *This ".class" file contains **Byte-Code's**.*
- *Bytecode is an intermediate code generated by java compiler which is neither understandable by computer nor the programmer.*
- *In order to run this byte-code's (Class File) we need to install an intermediate software known as JVM (Java Virtual Machine).*
- *JVM is used to convert bytecode into that respective machine-level language.*

NOTE:

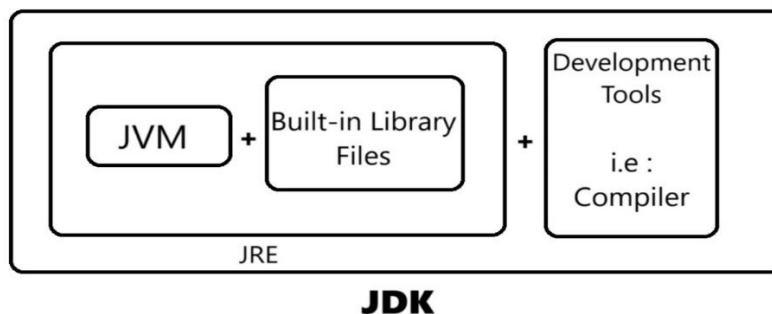
- *Creators of java has designed JVM's (JDK) for every system and as per respective hardware configurations.*
- *Hence, Java Proves the concept of platform in-dependency.*
- *So, Java software's are also called as write once Run anywhere software's.*

NOTE:

- *JVM (JDK) is platform dependent, but software's made using JDK are platform Independent.*



JDK Architecture:



1. JVM:

JVM is an interpreter which is used to convert bytecode to machine-level code line-by-line.

2. JRE:

JRE consists of JVM & Built-in library files which is used to run the java applications.

JRE is present inside JDK.

3. JDK:

JDK is a package given by Oracle Community which is used to develop & run Java applications.

JDK consist of JRE & development tools.

Steps to Create, Compile and Execute the java program

Step 1: To Create a java Program Editors and IDE's (Integrated Development Environment) are used

Editors: Notepad, Word, Notepad++, Edit-plus, Sublime, etc...

IDE: IntelliJ, Eclipse, VS-Code, Net-Beans

Step 2: Invoke the java Compiler to compile the java code.

Work of Java Compiler:

1. *Compiler checks for the syntax (Rules).*
2. *If the syntax is correct, class file is generated*
3. *If the syntax is wrong, class file is not generated and it will through Compile time error.*

Step 3: Then it calls the JVM (JRE) to run the class files.

Using Command Prompt:

1. *"dir": To explore the directory.*
2. *"cd_FolderName": Used to change the directory from parent folder to child folder.*
3. *"cls": Used to clear the Screen*
4. *"cd..": Used to get back to the Parent Folder.*
5. *"DriveName:": Shifting the Drives.*
6. *"mkdir" or "md": To create the Folder.*
7. *"javac -version": To check the version of java compiler.*
8. *"java -version": To check the JRE version.*
9. *"javac ClassName.java": Compiles the Program.*
10. *"java ClassName": Executes the Program.*
11. *"rmdir": To remove the folder.*

Structure of Java Program:

```
class ClassName
{
    public static void main(String[] args)
    {
        Statements;
        Statements;
    }
}
```

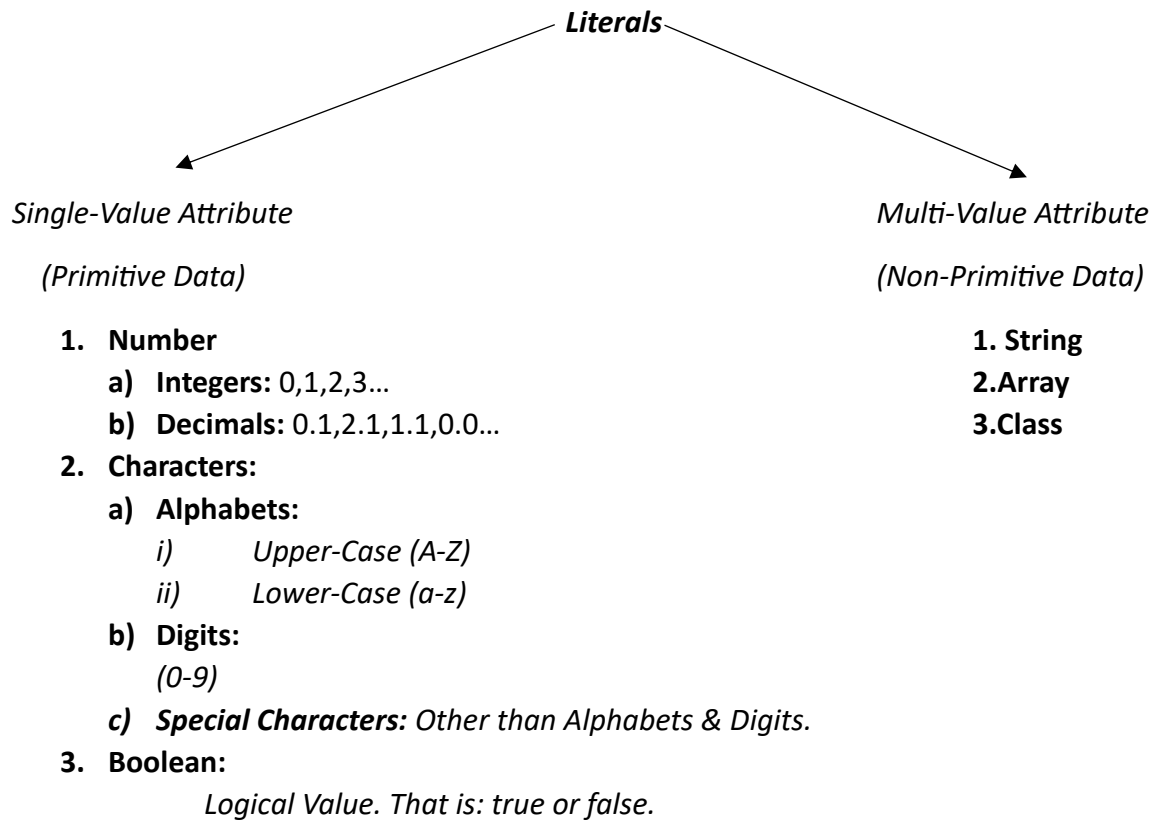
Class Block:

1. **Methods**
2. **Variables**

Syntax of Print Statement:

`System.out.println();` : Prints and ends the line.

`System.out.print();` : Prints and does not end the line.

Literals [Data]:**Primitive Data:**

1. **Numbers:**
 - (1) *Integers*
 - (2) *Decimals*
2. **Characters:**
 - Alphabets**
 1. *Uppercase*
 2. *Lowercase*
 - Digits**

0-9
 - Special Characters**

Other than Alphabets and Digits all are special characters
3. **Boolean:**

Logical Value, that is: true, false, etc...

Non-Primitive data:

1. *String (Group of characters are called String and always written in double quotes) i.e: "Vinay"*
2. *Array (Collection of Similar type of data)*
3. *Class*

Tokens:

- *Tokens are the smallest unit of programming language using which a programmer can create instructions.*

Tokens are classified into two types:

1. **Keywords**
2. **Identifiers**

1. Keywords:

1. *Compiler-aware words are called Keywords*
2. *Keywords have pre-defined meaning.*
3. *Keywords cannot be altered by a programmer.*
4. *Keywords are written in "lowercase".*
5. *There are more than 51 keywords in Java.*
6. *Keywords cannot be used for any other purpose*

Example: public, static, void, etc...

2. Identifiers:

- *Identifiers are the names given by the programmer to the members of the java, such as "class name", "method name", "variable name", "interface name", "package name" etc...*

Rules for Naming the Identifiers:

NOTE: Rules are mandatory to follow, if not followed we will get the compile time error.

Rule No. 1: *Identifier cannot start with the Number.*

Rule No. 2: *Character Spaces are not allowed while naming an identifier*

Rule No. 3: *Special characters are not allowed while naming an identifier except "\$" and "_".*

Rule No. 4: *We Cannot use keywords as identifiers.*

Conventions:

1. *Conventions are nothing but, the industry standards a programmer has to follow.*
2. *Conventions are not checked by Compiler but it is highly recommended to follow.*

Conventions for Class Name & Interface Name:

1. *If Class Name is single word, then the first character of the word must be in Upper-Case.*

Example: `Program1`

2. *If Class Name has Multiple words every first character of the word must be in Upper-Case.*

Example: `ThisIsMyFirstJavaProgram` (Pascal Case)

Conventions for Variable Name & Method Name:

1. *If the Method name is single word all the characters are written in lowercase.*

Example: `main()`

2. *If the Method name is multiple words the first character of first word will be in lowercase, from the second word every first character will be in Upper-Case.*

Example: `additionMethod()` (Camel Case)

Data Types:

Data Type helps the programmer to create containers (Variables) to store data.

Data Types are classified into Two types:

- 1. Primitive Data Types**
- 2. Non-Primitive Data Types**

1. Primitive Data Types:

Primitive data types help the programmer to create primitive type containers (Primitive Variable) to store primitive data.

There are 8 Primitive Data Types in Java

Primitive Data	Primitive Data Types	Maximum Size	Default Value
Integer	Byte	1 Byte	0
	Short	2 Bytes	0
	Int	4 Bytes	0
	Long	8 Bytes	0L / 0L
Decimal	Float	4 Bytes	0.0f / 0.0F
	Double	8 Bytes	0.0 / 0.0D
Character	Char	2 Bytes	������
Boolean	Boolean	1 Bit	False

Variables:

Variables are the containers inside which we can store the data.

IMPORTANT NOTE:

- We can create a variable with the help of the datatype, without datatype we cannot create a variable.
- In java, we cannot declare the variable without declaring it.

Variable Declaration Statement:

Declaring the datatype to the variable is called Variable Declaration Statement.

Syntax: datatype variableName;

Example: `String str;`
`Int a;`

Variable Initialization Statement:

Assigning value to the variable is called Variable Initialization Statement

Syntax: `variableName = value/expression/another variable;`

Example: `str = "Vinay"`
`a = 7;`

Variable Declaration and Initialization Statement:

Here, we can declare the variable and store the data simultaneously.

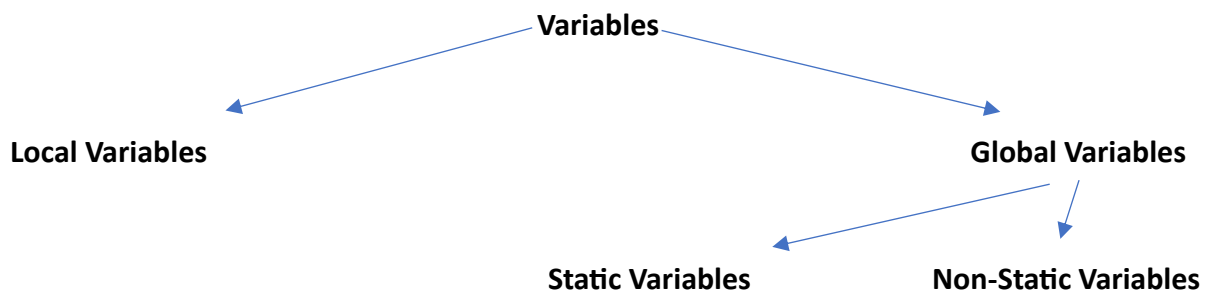
Syntax: `datatype variableName = value/expression/another variable;`

Example: `String str = "Vinay";`
`Int a = 7;`

Example Program:

```
class Introduction
{
    public static void main (String [] args)
    {
        String name = "Vinay Nalavade";
        byte age = 22;
        float cgpa = 7.95f;
        double salary = 2500000.00;
        long phone = 9359645433L;
        char grade = 'A';

        System.out.println(name);
        System.out.println(age);
        System.out.println(cgpa);
        System.out.println(salary);
        System.out.println(phone);
        System.out.println(grade);
    }
}
```

Classification of Variables:

Variables are classified into Two Types:

1. *Local Variables*
2. *Global Variables*

Global Variables:

The Variables which are declared in the class block are called Global Variables.

Global Variables are classified into Two Types:

1. *Static Variables*
2. *Non-Static Variables*

Local Variables:

The Variables which are declared in any other block other than class block are called Local Variables.

Characteristics of Local Variable:

1. *A Variable declared inside a block remains local to that block & it cannot be accessed outside of the block.*

Example:

```
class LocVar1 {  
    public static void main (String[]args) {  
        {  
            Double d = 234.234;  
        }  
        System.out.println(d);  
    }  
}
```

Output: Compile Time Error.

2. *We cannot have more than one Local Variable of same name with-in same block Otherwise, we will get Compile Time Error*

Example:

```
class LocVar1 {
```

```
public static void main(String[]args) {  
    {  
        Double d = 234.234;  
        System.out.println(d);  
        String d = "Vinay";  
        System.out.println(d);  
    }  
}
```

Output: Compile Time Error

3. *But We can have more than one Local Variable of same name in two different blocks.*

Example:

```
class LocVar1 {  
    public static void main(String[]args) {  
        {  
            Double d = 234.234;  
            System.out.println(d);  
        }  
        String d = "Vinay";  
        System.out.println(d);  
    }  
}
```

Output: 234.234

Vinay

4. *We cannot use a Local Variable without initialization, because Local Variables are not given with default values.*

Example:

```
class LocVar1  
{  
    public static void main(String[]args)  
    {  
        boolean b;  
        System.out.println(b);  
    }  
}
```

Output: Compile Time Error

Operators:

- *Operators are the symbols which are used to perform certain operations on the given values or operands.*

Operators are classified in Two Types:

1. *Based on Number of Operands*
2. *Based on Type of Operands*

Based on Number of Operands Operators are classified as:

1. *Unary Operators: Need 1 Operand*
2. *Binary Operators: Need 2 Operands*
3. *Ternary Operators: Need 3 Operands*

Based on Operations it performs are classified as:

1. *Arithmetic Operators*
2. *Assignment Operators*
3. *Relational Operators*
4. *Conditional Operators*
5. *Logical Operators*
6. *Bitwise Operators*
7. *Increment & Decrement Operators*
8. *Compound Assignment Operators*
9. *Miscellaneous Operators*

A. Arithmetic Operators:

- *Arithmetic Operators are Binary operators which are used to perform Basic Mathematical Calculations.*

Such as –

- a. *Addition (+)*
- b. *Subtraction (-)*
- c. *Multiplication (*)*
- d. *Division (/)*
- e. *Modulus (%)*

a. Addition:

Operand 1	Operator	Operand 2	Result	Type
5 (byte)	+	10 (byte)	15	Int
20 (short)	+	30 (short)	50	Int
'a' (char)	+	'b' (char)	195	Int
30 (int)	+	20 (int)	50	Int
50 ℓ (long)	+	100 ℓ (long)	150	Long
1.1 f (float)	+	2.2 f (float)	3.3	Float
1.0 (double)	+	3.2 (double)	4.2	Double
"Hello" (String)	+	"World" (String)	Hello World	String
True (Boolean)	+	True (Boolean)	CTE	CTE

b. Subtraction:

Operand 1	Operator	Operand 2	Result	Type
4 (byte)	-	3 (byte)	1	Int
20 (short)	-	12 (short)	8	Int
'M' (char)	-	'a' (char)	-20	Int
100 (int)	-	35 (int)	65	Int
66 ℓ (long)	-	62 ℓ (long)	4	Long
21.4 f (float)	-	18.9 f (float)	2.5	Float
71.1 (double)	-	70.2 (double)	0.899	Double
"Hello" (String)	-	"World" (String)	Bad Operand	CTE
True (Boolean)	-	True (Boolean)	CTE	CTE

c. Multiplication:

Operand 1	Operator	Operand 2	Result	Type
6 (byte)	*	10 (byte)	12	int
21 (short)	*	2 (short)	42	Int
'O' (char)	*	'N' (char)	6162	Int
13 (int)	*	3 (int)	231	int
77 ℓ (long)	*	3 ℓ (long)	150	Long
1.6 f (float)	*	2.1 f (float)	3.36	Float
1.44 (double)	*	2.1 (double)	3.024	Double
"Hello" (String)	*	"World" (String)	Bad Operand	CTE
True (Boolean)	*	True (Boolean)	CTE	CTE

d. Division:

Operand 1	Operator	Operand 2	Result	Type
72 (byte)	/	6 (byte)	12	int
61 (short)	/	20 (short)	3	Int
'X' (char)	/	'U' (char)	1.03	Int
12 (int)	/	6 (int)	2	int
72 ℓ (long)	/	2 ℓ (long)	36	Long
2.31 f (float)	/	1.6 f (float)	1.44	Float
16.4 (double)	/	10.4 (double)	1.57	Double
"Hello" (String)	/	"World" (String)	Bad Operand	CTE
True (Boolean)	/	True (Boolean)	CTE	CTE

e. Modulus:

Operand 1	Operator	Operand 2	Result	Type
31 (byte)	%	6 (byte)	1	int
62 (short)	%	2 (short)	0	Int
'B' (char)	%	'A' (char)	1	Int
60 (int)	%	20 (int)	0	Int
10 ℓ (long)	%	71 ℓ (long)	10	Long
1.62 f (float)	%	1.71 f (float)	1.62	Float
70.4 (double)	%	17.7 (double)	17.300	Double
"Hello" (String)	%	"World" (String)	Bad operand	CTE
True (Boolean)	%	True (Boolean)	CTE	CTE

Key-Board Symbols with Name:

Sr.No	Symbols	Name
1	`	Back Quote
2	~	Tilde
3	!	Exclamation point
4	@	At- sign
5	#	Hash-tag
6	\$	Dollar sign
7	%	Percent
8	^	carat
9	&	And-sign, Ampersand
10	*	Asterisks
11	(	Open-parenthesis
12	)	Close-parenthesis
13	-	Dash, minus-sign
14	_	Underscore
15	\	Back-slash
16		Pipe, Bar
17	;	Semi-colon
18	:	Colon
19	'	Apostrophe, Single-quote
20	"	Double-quote
21	<	Less-than, Open angle Bracket
22	>	Greater-than, Close angle bracket
23	/	Slash
24	?	Question Mark

B. Relational Operator:

They are Binary Operators. Relational Operators are used to Compare two values or operands for decision making purpose.

Such as –

- a. Equality (==)
- b. Greater-than (>)
- c. Lesser-than (<)
- d. Less than or equal to (<=)
- e. Greater than or equal to (>=)
- f. Not Equal (!=)

Note: The Result type of the Relational Operator is Boolean.

Example:

```
class Relations
{
    public static void main (String [] args)
    {
        System.out.println(3 < 4);
        System.out.println('a' > 'A');
        System.out.println(0.0 > 0);
        System.out.println('a' < 113);
        System.out.println(true > false);
        System.out.println(59.9999 == 60);
        System.out.println(true == false);
        System.out.println(true != false);
        System.out.println(byte < short);
        System.out.println(true == 1);
        System.out.println("Hello" < "Hi");
        System.out.println("Hi" != "Bye");
        System.out.println("HI" == "hi");
        System.out.println(1.1 < 1.1);
        System.out.println((22.0/7.0) > (22.0f/7.0));
    }
}
```

INTERVIEW QUESTIONS:

For Testing Engineer

1. Difference between selenium ID and Web Driver?
2. Difference between Functional & Non-functional Testing?
3. Features of Java?
4. Difference between JDK, JRE & JVM?
5. What is Abstract Class and Abstract Keyword?
6. What is Static Keyword?
7. Difference between Equality Operator and Equal Method?
8. What is Method Overloading and Method Overriding?
9. Explain Finally Block?
10. Explain Constructors?
11. Difference between "This" and "Super" keyword?
12. What is Verification and Validation in Testing?
13. What is Bug lifecycle?
14. What are the API testing tools?
15. Explain OOP's concept?
16. Check given string is Palindrome or not?
17. Find the highest element of given array?

Assignment Operator:

It is the **Binary Operator** which is used to store a value inside a variable.

Rule:

1. LHS of an assignment operator must be a variable.
2. RHS can be a value or an expression or another variable.

Syntax:

Variable = value / expression / another variable;

LHS = RHS

The direction of execution of assignment operator is from Right to Left.

Example:

`Int a = 10;`

In Assignment operator the type of value to be stored must be same as type of variable provided.

Example:

`String S = True;        // Compile-time Error`

`Boolean B = true;`

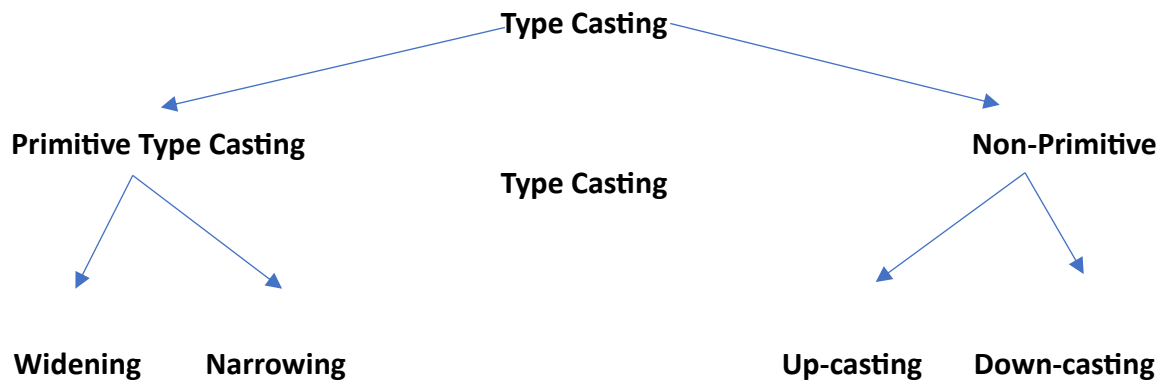
If the type of value is not same as the type of variable provided, there are 2 possibilities.

- If the type of value is not same with the type of variable there are 2 possibilities.
- If the type of value is smaller than type of variable the compiler performs widening.
- If the type of value is larger than type of variable, we get CTE (Narrowing is not done Implicitly)

Error Name: Possible Lossy Conversion.

Type-Casting:

Type casting is a process of converting or storing one type of data into another type is known as Type casting.

**Primitive Typecasting:**

It is process of converting or storing one type of primitive data into another primitive type is known as Primitive typecasting.

It is classified into two types:

1. Widening
2. Narrowing

Ascending Order by Data-types:

Byte < Short < Char < Int < Long < Double

Note: Short and Char have same size of 2 bytes but Char occupies more space than Short.

A. Widening:

- It is a process of converting or storing smaller type value in large type of variable.
- Widening is done implicitly by the compiler as there is NO loss of data.

B. Narrowing:

- Narrowing is the process of converting or storing larger type of data in smaller type.
- Narrowing is not done implicitly by compiler because there would be the loss of data.
- But Narrowing can be done explicitly by the programmer with help of typecast operator.

Syntax:

= (type to convert) value / Expression
 int i = (int) 10.05;

Example:

```

class Narrowing {
    public static void main (String [] args) {
        int i = (int) 2345.345423f;
        System.out.println(i);
        char c = (char) 3123.234234;
        System.out.println(c);
        byte b = (byte) 100000;
        System.out.println(b);
    }
}
  
```

```

        System.out.println(10+(int)2343.2343+'a');
    }
}

```

Output:

```

2345
?
-96
2450

```

Que. A Man has to cover a distance of 1.7236 Km So, he has plans to walk for given distance please help him by calculating the number of steps he needs to cover distance?

➔ *Step Distance = 0.834 Km*

Distance = 1.7623 Km

*= 1.7623 * 1000*

= 1762.3 Km

Steps = Distance / Distance of Steps

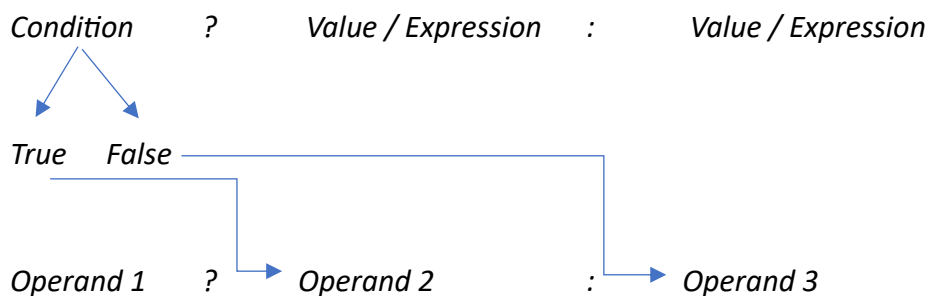
= 1762.3 / 0.834

= 2113 m.

Conditional Operators:

It is a **Ternary Operator** which is used for Decision making purpose.

Syntax:



Example:

```

class Conditional {
    public static void main (String [] args) {
        int n1 = 200;
        int n2 = 50;
        int ans = n1 > n2 ? n1 : n2;
        System.out.println("The Largest of "+n1+" and "+n2+" is: " +ans);
    }
}

```

- The Operand 1 must be a Condition and it must be of Boolean type.
- If Condition is evaluated to true the control goes to Operand 2.
- If Condition is evaluated to false the control goes to Operand 3.
- The Result type of a Conditional operator depends on the type of value present in Operand 2 and Operand 3.

Compound-Assignment Operator:

Compound - Assignment operators are **Binary Operators** which are used to update the value in a variable.

Compound – Assignment operators are also called Update Operators.

Syntax:

Variable += Value / Expression;

Example:

```
class Compound {
    public static void main (String [] args) {
        double sal = 5000;
        sal += 10000;
        System.out.println(sal);
    }
}
```

Output: 15000

Logical Operator:

- Logical Operators are used for logical decision-making purpose.

Logical Operator are:

- i. **AND**
- ii. **OR**
- iii. **NOT**

1. AND Operator (&&):

- It is a **Binary Operator**.

Syntax:

Operand 1 && Operand 2

Truth Table:

Operand 1	Operand 2	Result
True	False	False
False	True	False
False	False	False
True	True	True

Working:

The Operands must be **Boolean Type**.

If any operand is **false** then result of **AND** operator is **false**.

2. OR Operator (||):

It is a **Binary Operator**.

Syntax:

Operand 1 || operand 2

Truth Table:

Operand 1	Operand 2	Result
True	False	True
False	True	True
False	False	False
True	True	True

Working:

The Operands must be **Boolean Type**.

If any operand is **true** then result of **OR** operator is **true**.

3. NOT Operator (!):

It is a **Unary Operator**.

Syntax:

! Operand 1

Truth Table:

Operand	Result
True	False
False	True

Working:

The Operands must be **Boolean Type**.

If operand is **true** then result is **false** & visa-versa.

Increment Operator & Decrement Operator:

- It is an **Update Operator**.

1. Increment Operator:

Types of Increment Operator:

- Pre-Increment Operator
- Post-Increment Operator

a. Pre-Increment Operator:

- It is a **Unary Operator**
- If a variable is prefixed with Increment Operator, then it is known as Pre-Increment Operator.

Syntax:

++variable;

Working:

- The old value in variable is added by one value and then updated value is used / substituted.

b. Post-Increment Operator:

- It is a **Unary Operator**
- If a variable is Suffixed with Increment Operator, then it is known Post-Increment Operator.

Syntax:

Variable++;

Working:

- The value in the variable is first substituted and variable is added with one value.

2. Decrement Operator:**Types of Decrement Operator:**

- a. Pre-Decrement Operator:
- b. Post-Decrement Operator:

a. Pre-Decrement Operator:

- i. It is a **Unary Operator**
- ii. If a variable is prefixed with Decrement Operator, then it is known as Pre-Decrement Operator.

Syntax:

--variable;

Working:

- The old value in variable is subtracted by one value and then updated value is used / substituted.

b. Post-Decrement Operator:

- i. It is a **Unary Operator**
- ii. If a variable is Suffixed with Decrement Operator, then it is known as Post-Decrement Operator.

Syntax:

Variable--;

Working:

- The value in the variable is first substituted and variable is subtracted with one value.

Reading Data Dynamically from the User:

- We can read data dynamically (at the runtime) from the user with the help of a built-in class known as **Scanner Class**.
- Scanner Class is a built-in class present in **java.util Package**.

Steps to Use Scanner Class:

1. Import Scanner class from java.util package.
 - a. Using Import keyword we can import built-in members.
 - b. Import statement must be the first statement in a java program.

Syntax:

Import java.util.Scanner;

2. Create an Object of Scanner class with Scanner Constructor and store it's reference in a Scanner Variable.

Syntax:

Scanner sc = new Scanner (System.in);

3. Using Scanner reference call the Scanner class to read data dynamically from the user respectively.

<i>Data Type</i>	<i>Method Signature</i>
Byte	nextByte()
Short	nextShort()
Int	nextInt()
Long	nextLong()
Float	nextFloat()
Char	next().charAt(0)
Double	nextDouble()
Boolean	nextBoolean()
String	next() / nextLine()

Operator Precedence:

<i>Level</i>	<i>Operator</i>	<i>Description</i>	<i>Associativity</i>
16	() [] .	Parentheses Array Access Member Access	Left – to – Right
15	++ --	Unary Post-Increment Unary Post-Decrement	Left – to – Right
14	+ - ! ~ ++ --	Unary Plus Unary Minus Unary Logical NOT Unary Bitwise NOT Unary Pre-Increment Unary Pre-Decrement	Right – to – Left
13	() New	Cast Object Creation	Right – to – Left
12	* / %	Multiplicative	Left – to – Right
11	+ - +	Additive String Concatenation	Left – to – Right
10	<< >> >>>	Shift	Left – to – Right
9	< <= > >=	Relational	Left – to – Right
8	== !=	Equality	Left – to – Right
7	&	Bitwise AND	Left – to – Right
6	^	Bitwise XOR	Left – to – Right
5		Bitwise OR	Left – to – Right
4	&&	Logical AND	Left – to – Right
3		Logical OR	Left – to – Right
2	?:	Ternary	Right – to – Left
1	= += -= *= ^= /= <<= >>= >>>=	Assignment	Right – to – Left
0	->	Lambda Expression Arrow	Right – to – left

Note:

- **Jumping** is happening because of return in **Buffer Memory**.

Control Flow Statements:

- The Statements which are used to control the flow of the execution of the program.

Control-Flow Statement are classified into 2 types:

A. Decision Statements

B. Loop Statements

A. Decision Statements:

- **Decision Statement** helps the programmer to decide whether to execute a set of instructions or skip them.

There are 4 Decision Statements:

1. If Statement
2. If- Else Statement
3. If- Else- If Ladder
4. Switch Statement

1. If Statement:

Workflow:

- a. if Statement consist of a condition and a block with statements.
- b. The condition must be Boolean type.
- c. If condition evaluates to **False** the statements of the block will be skipped.
- d. If condition evaluates to **True** the statements of the block gets executed.

2. If-Else Statement:

Workflow:

- a. An If-Else statement consist of "**IF**" block with a condition and "**ELSE**" block where we put statements.
- b. If a condition is evaluated to **TRUE** the statements of "**IF**" block executes and "**ELSE**" block gets skipped.
- c. If a condition is evaluated to **FALSE** the statements of "**IF**" gets skipped and "**ELSE**" block gets executed.

NOTE:

In **IF-ELSE** Statement only one block gets executed at a time [Either **IF** or **ELSE**].

3. IF-ELSE-IF Ladder:

Workflow:

- a. In an **If-Else-If** ladder we have **If** block followed by **Else-If** blocks and then **Else** block.
- b. **If-Else-If** ladder is used to check **Multiple Conditions**.
- c. if the condition in **If** block becomes **true** the statements of that block get executed and control is **transferred** outside of the **Else-If Ladder**.
- d. If the condition becomes **False** the control **shifts** to next condition, this keeps on happening until any one condition becomes **true**.

- e. If none of conditions become **true** control goes to **Else** block [if provided] and statements of **Else** block gets executed.
- f. If there is **no Else Block** then it executes **Nothing**.

NOTE:

- i. Having **Else Block** is Optional.
- ii. If **Multiple** Conditions are **true**, the **first condition** which becomes **true**, that block gets executed.
- iii. In **If-Else-If Ladder** at-most **one** block gets executed at a time.

4. Switch Statement:**Workflow:**

- a. In a Switch Statement we have a **Switch Block** which can accept a **Value** or an **Expression** or a **Variable**.
- b. A **Switch Block** consist of Multiple Case Blocks which can accept **Value** or **Expression**.
- c. If Value present in a **Switch** block **match** with a value present in a **case block**, then the statements of that case block get **executed** and if a **break** statement is provided, the control transferred **outside** of the **Switch Block**.
- d. If **Break** is not used the rest other **all** case blocks **executed** without checking the condition including the **default block** [if provided].
- e. If the value in the **Switch block** does **not match** with value in a **case block**, the control transfers for the next case block, this keeps on happening until any **one case matches**.
- f. If none of the case blocks matches then **default block** get executed [if provided] or else it executes **Nothing**.

NOTE:

- i. **Variables** are **not allowed** with **case** blocks as case block must have **constant** Values.
- ii. **Data Type** such as **Long, Float, Double, Boolean** are **not allowed** in Switch Statement.
- iii. **Data Type** such as **Byte, Short, Int, Char** are **allowed** in Switch Statement.
- iv. Using **Default Statement** in Switch case is **Optional**.

Break () Statement:

- i. **Break** is a **Keyword** and a control transfer Statement.
- ii. It is also known as **Jump** Statement.
- iii. **Break** Statement can be used inside the **Switch** Statements & **Loop** Statements.
- iv. if **Break** Statement executes, the control bought outside the **Switch** or **Loop** Statement (Stops the execution of the rest of the Statement).
- v. In **Switch Statement** we can use **Multiple cases** but, we **cannot** use it in **If-Else-If Ladder**.

B. Loop Statements:

- **Loop Statements** are the component in java which are used to execute the set of instructions repeatedly.

Every Loop Statement has 3 Parts:

1. Initialization

- It specifies the **Starting Point** of the Loop

2. Condition

- It is used to control the execution of the Loop.

3. Update

- It is used to make the condition **False** / **Break** the Condition.

There are 4 Loop Statements:

1. While Loop
2. Do-While Loop
3. For Loop
4. For-Each Loop / Advanced For / Enhanced For Loop

1. While Loop:

- It is a Loop Statement.

Workflow:

- a. A **While Loop** is used to execute a set of instructions repeatedly, where the **initialization** is done **before** the starting of the loop.
- b. In the loop we should provide a **condition** which is of **Boolean type**.
- c. If a condition is **True** the statements of the block gets executed.
- d. Once all the statements are executed **Update Statement** is given.
- e. Once after the **Update** condition is rechecked to decide whether to **continue** the process (re-do the task) or to **Stop**.
- f. This keeps on repeating until the condition becomes **False**.

2. Do-While Loop:

- It is a Loop Statement.

Workflow:

- a. In a **Do-While Loop** **first** the Statement in **Do** block will be executed & then the condition will be checked.
- b. If the condition is not satisfied the process repeats until the condition becomes **False**.
- c. The **Initialization** must be done **before** the declaration of the **Do-While Loop**.
- d. Condition must be of **Boolean type**.

NOTE:

Minimum number of executions of a **Do-While Loop** is **One**. But, minimum number of executions of **While Loop** is **Zero**.

3. For Loop:

- It is a Loop Statement.

Workflow:

- In a **For Loop** the **Initialization**, **Condition** and the **Update** are Declared together with a semi-colon separation.
- First**, the control goes to **Initialization** part & then to the **Condition**.
- If a **condition** is evaluated to **True** the statements of Loop Block get Executed.
- Once all the statements are executed the control goes to the **Update** part and Updates.
- Once after the **Update** the control goes **back** to the **Condition** and the **Cycle Repeats** until the condition becomes **False**.

Leap Year Program:

Actual Year Length is **365.2450 Days**. But, we **rounded** it to **365.25 Days**.

So,

0.25 stands for **6 Hours**.

365.25 x 4 Years = 1 Day Extra.

We added that extra day into **4th Year**.

But,

There is still error exist.

We **rounded** the **days** but still it gives the error of **0.0075 days**, that means something around **18 minutes**.

So,

We checked how it will matter in **100 years**

100 years x 0.0075 days = 0.75 days

200 years x 0.0075 days = 1.50 days

300 years x 0.0075 days = 2.25 days

400 years x 0.0075 days = 3 days

In, **400 years** it gave **3 extra days**.

So,

We removed that extra **3 days** from the **100th year** for next **300 years**.

So,

It came up with the **condition** of **leap year** as,

If ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))

Number Programs:**Prime Number Program:**

- Divisible by 1 or itself only (1 is not a prime Number).

- **Program:**

```
import java.util.Scanner;
class Prime
{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter Number: ");
        int num = sc.nextInt();
        int den = 2;
        while (den <= num)
        {
            if (num%den == 0)
            {
                break;
            }
            den++;
        }
        if (num == den)
        {
            System.out.println("Number is Prime Number");
        }
        else
        {
            System.out.println("Number is not a Prime Number");
        }
    }
}
```

Fibonacci Series Program:

- Starts from 0 & 1 and further increase with addition of two consecutive numbers generated before.

- **Program:**

```
import java.util.Scanner;
class Fibonacci
{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
```

```

        System.out.print("Enter how much Fibonacci Values you want : ");
        int n = sc.nextInt();
        int n1 = 0, n2 = 1, n3;
        for (int i = 0; i < n ; i++ )
        {
            n3 = n1 + n2;
            System.out.print(n1+",");
            n1 = n2;
            n2 = n3;
        }
    }
}

```

Least Common Multiple:

- Identifying the smallest common divisible number for both numbers given by user.

- **Program:**

```

class LCM
{
    public static void main (String [] args)
    {
        System.out.println("---Least Common Multiple Program---");
        int n1 = 18;
        int n2 = 18;
        int big = n1 > n2 ? n1 : n2;
        int i = 1;
        while (true)
        {
            if ((big * i) % n1 == 0 && (big * i) % n2 == 0)
            {
                System.out.print("LCM of "+n1+" & "+n2+" is: "+big*i);
                break;
            }
            i++;
        }
    }
}

```

Co-Prime Numbers Program:

- There should not be any other divisor for numbers other than 1. In other words, there GCD must be 1. (ex. 15, 28)

- **Program:**

```

import java.util.Scanner;
class CoPrimeNum

```

```

{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter First Number : ");
        int n1 = sc.nextInt();
        System.out.print("Enter Second Number : ");
        int n2 = sc.nextInt();
        int small = n1 < n2 ? n1 : n2;
        while (true)
        {
            if (n1 % small == 0 && n2 % small == 0)
            {
                break;
            }
            small--;
        }
        if (small == 1)
        {
            System.out.print(n1+" & "+n2+" are Co-Prime Numbers");
        }
        else
        {
            System.out.print(n1+" & "+n2+" are NOT Co-Prime Numbers");
        }
    }
}

```

Buzz Number:

- A number which ends with 7 or Divisible by 7
- **Program:**

```

import java.util.Scanner;
class BuzzNum
{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter The Number : ");
        int num = sc.nextInt();
        if (num % 7 == 0 || num % 10 == 7)
        {
            System.out.print(num+" is a Buzz Number");
        }
    }
}

```

```

        else
        {
            System.out.print(num+" is NOT a Buzz Number");
        }
    }
}

```

Neon Number:

- sum of the digits of the square of number is equal to that number itself (ex. $9^2 = 81$, $8+1 = 9$)

- **Program:**

```

import java.util.Scanner;
class NeonNum
{
    public static void main (String [] args)
    {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter the Number : ");
        int n1 = sc.nextInt();
        int n2 = n1 * n1;
        int sum = 0;
        while (n2 > 0)
        {
            int mod = n2 % 10;
            n2 /= 10;
            sum = sum + mod;
        }
        if (sum == n1)
        {
            System.out.print("Neon");
        }
        else
        {
            System.out.print("Not Neon");
        }
    }
}

```

Pronic Number, Oblong Number, Heterogenic Number:

- Product of two consecutive integers. (ex. $2*3 = 6$)

- **Program:**

```

import java.util.Scanner;
class PronicNum
{

```



```

public static void main (String [] args)
{
    System.out.print("Enter the Number: ");
    Scanner sc = new Scanner (System.in);
    int n1 = sc.nextInt();
    int n = 1;
    for ( n = n ; n1 > n ; n++ )
    {
        if (n*(n+1) == n1)
        {
            break;
        }
    }
    if (n1 == n)
    {
        System.out.println(n1+" is Not Pronic Number");
    }
    else
    {
        System.out.println(n1+" is Pronic Number");
    }
}
}

```

Strong Number, Peterson Number:

- Sum of Factorial of individual digits. (ex. 145)
- **Program:**

```

class StrongNumber
{
    public static void main (String [] args)
    {
        int n1 = 145;
        int sum = 0;
        while (n1 != 0)
        {
            int mod = n1 % 10;
            int fact = 1;
            for (int i = mod ; i > 0 ; i-- )
            {
                fact = fact * i;
            }
            sum = sum + fact;
            n1 /= 10;
        }
    }
}

```

```

    }
    System.out.print("Sum Of all Factorial Digits : "+sum);
}
}

```

Nested Loops:

- A Loop declared inside another loop is known as Nested Loop.
- Nested Loops are used when we want to do a repetitive task repeatedly.

Syntax:

```

class NestedLoops
{
    public static void main (String [] args) {
        for (int; Condition; Update) {
            for (int; Condition; Update) {
                //Statements;
            }
            //Statements;
        }
        //Statements
    }
}

```

Arm-Strong Number, Narcissistic Number, Pluperfect Number:

- Sum of its own digits each raised to the power of number of digits. (ex. $1^3 + 5^3 + 3^3 = 153$).

- Program:

```

import java.util.Scanner;
class ArmStrong
{
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter Number to check ArmStrong: ");
        int num = sc.nextInt();
        int count = 0;
        int sum = 0;
        for (int i = num; i != 0 ; i/=10) {
            count++;
        }
        for (int i = num; i != 0 ; i/=10 ) {
            int last = i % 10;
            int pow = 1;
            for (int j = 0; j < count; j++) {
                pow = pow * last;
            }
            sum += pow;
        }
        System.out.print("Sum of digits raised to the power of number of digits: "+sum);
    }
}

```

```

        }
        sum = sum + pow;
    }
    if (sum == num) {
        System.out.print("ArmStrong Number");
    }
    else {
        System.out.print("NOT ArmStrong Number");
    }
}
}

```

Duck Number:

- A Number that has zeroes present in it, but not at the beginning of the Number. (ex. 1203)

- **Program:**

```

class DuckNumber
{
    public static void main (String [] args)
    {
        int num = 1230;
        while (num != 0)
        {
            int last = num%10;
            if (last == 0)
            {
                break;
            }
            num/=10;
        }
        if (num != 0)
        {
            System.out.println("It is a Duck Number");
        }
        else
        {
            System.out.println("It is NOT a Duck Number");
        }
    }
}

```

Pattern Programs:**Zig-Zag Number Pattern:**

```

package PatternPrograms;

public class NumbersZigZag {
    public static void main (String [] args)
    {
        int n = 5;
        int incOdd = 7, incEven = 1, ans;
        for (int i = 1; i <= n ; i++)
        {
            ans = i;
            System.out.print(ans);
            for (int j = 1; j <= n; j++)
            {
                if ( j % 2 == 0)
                {
                    ans = ans + incOdd;
                    System.out.print(ans+ " ");
                }
                else
                {
                    ans = ans + incEven;
                    System.out.print(ans+ " ");
                }
            }
            System.out.println();
            incOdd -= 2;
            incEven += 2;
        }
    }
}

```

Output:

```

12 9 10 17 18
25 10 13 18 21
38 11 16 19 24
41 12 19 20 27
51 13 22 21 30

```

Straight Numbers:

```

package PatternPrograms;

public class NumbersColumnByColumn {

```

```

public static void main (String [] args)
{
    int n1 = 10;
    for (int i = 1; i < n1 ; i++ )
    {
        for (int j = 1; j < n1 ;j++ )
        {
            System.out.print(j + " ");
        }
        System.out.println();
    }
}

```

Output:

```

1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9

```

Alphabets in Square Pattern:

```

package PatternPrograms;

public class AlphabetsSquare {
    public static void main (String [] args)
    {
        char n = 'a';
        for (int i = 1; i < 5 ;i++ )
        {
            for (int j = 1; j < 5 ;j++ )
            {
                System.out.print(n+" ");
                n++;
            }
            System.out.println();
        }
    }
}

```

Output:

```

a b c d
e f g h
i j k l
m n o p

```

Diagonal Numbers Pattern:

```

package PatternPrograms;

public class DiagonalNumbers {
    public static void main (String [] args)
    {
        int n = 5;
        for (int i = 1; i < n ; i++ )
        {
            for (int j = 1; j < n ; j++ )
            {
                if (i == j)
                {
                    System.out.print(j + " ");
                }
                else
                {
                    System.out.print("*"+" ");
                }
            }
            System.out.println();
        }
    }
}

```

Output:

```

1 * * *
* 2 * *
* * 3 *
* * * 4

```

RAT Pattern:

```

package PatternPrograms;

public class RatPattern {
    public static void main (String [] args)
    {
        int n = 10;
        for (int i = 1; i < n; i++ )
        {

```

```
        for (int j = 1; j < n ; j++ )
        {
            if (i == 1 || i+j == n      || i == j || i == n-1)
            {
                System.out.print("*"+" ");
            }
            else
            {
                System.out.print(" "+" ");
            }
        }
        System.out.println();
    }
}
```

Output:

```
* * * * *
*       *
*     *
*   *
* *
* *
*   *
*     *
*       *
* * * * *
```

Method:

- Method is a block of instruction which is designed to perform a specific task.
- Methods are used to achieve **Code Modularity**.

Code Modularity:

- It is a design of breaking entire task into smaller modules and sub-modules and creating specific methods for each and every module is known as Code Modularity.

Advantages of Code Modularity:

1. Code Reusability
2. Easy Code Modification
3. Easy Bug Fixing
4. Readability of Code Increases.

Types of Methods:

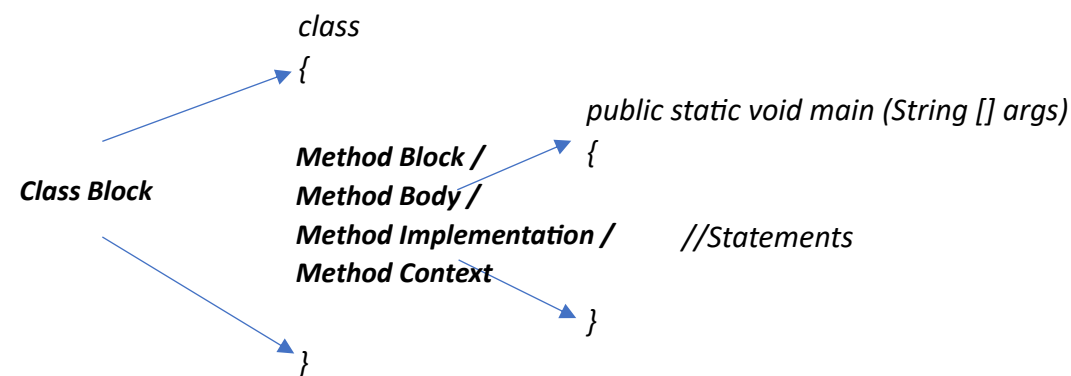
1. User-Defined Methods
2. Built-in Methods

1. Built-in Methods:

- The Methods which are created by creators of language or framework. Which are available (can be used) directly.
- Example: **Scanner Class Method**

2. User-Defined Methods:

- A Programmer can also design and create his own methods those methods can be used by himself or someone else.

Syntax of Methods:

public: Access Modifier

static: Modifier

void: Return type

main: Method Name

(String [] args): Formal Arguments

Formal Arguments:

- *Formal Arguments are the Variables which are declared in method declaration statement.*
- *Formal arguments are required if the method has to accept the input data.*

Method Name:

- *It is an identifier, which is used to identify and call the method for execution.*
- *Follow the conventions for naming the method.*

Return Type:

- *It Specifies whether the method will return some value at the end of the execution or it returns nothing.*
- *Return type can be either void or any specific data-type.*
- *If Method returns nothing the return type of method is void.*
- *If Method returns something then return type of method must that specific data-type.*

Modifier:

- *Modifiers are the key-words which are used to alter the behaviour of a java members.*
- *They are classified into Two Types:*
 - Access Modifiers*
 - Non-Access Modifiers*

Access-Modifiers:

- *These are the key-words which alter the Behaviour of the Java Member according to the accessibility or visibility.*
- *They are:*
 - Public*
 - Private*
 - Protected*
 - Default*

Non-Access Modifiers:

- *They are the Keywords.*
- *They are:*
 - Static*
 - Final*
 - Abstract*
 - Volatile*
 - Synchronized*
 - Etc...*

Method Block:

- *It is a block where java Statements are written step-by-step to perform the specific task.*

Important Terminologies:**1. Method Signature:**

- It is a combination of Method Name & Formal Arguments
- Example: **main (String [] args)**

2. Method Declaration:

- It is a Combination of Access Modifier, Modifier, Return type and Method Signature
- Example: **public static void main (String [] args)**

3. Method Definition:

- It is a combination of Method Declaration and Method body.
- Example: **public static void main (String [] args) { //Method Body }**

Types of Methods:

- Methods are classified into two types based on the number of arguments it can accept.
 1. No-Argument Method
 2. Parameterized Method

1. No-Argument Method:

- Methods without formal arguments are known as No-Argument Method.
- No-Argument Methods are designed when the method doesn't want to accept any input data.
- These methods are designed when we want to execute standard set of instructions.

Characteristics of Methods:

- A Method gets executed only when it is called.
- We can call a method using a method call statement.
- A method can be called any number of times.
- A variable declared inside the method block will remain local to that Method, we cannot use it in different method.
- A class can have any Number of Methods.

2. Parameterized Methods:

- Methods with formal arguments are called parameterized methods.
- It is used to accept input values when called.

Formal Arguments:

- The variables in the method declaration statement are known as formal arguments.
- Formal arguments are the local variables of the methods.
- Formal arguments will be initialized with data when method is called for execution.

Actual Arguments:

- It is a data or a value which are passed in a method call statement.
- Actual Arguments can also be an expression, where the expressions are resolved first and the result is passed in the methods.

Binding Process of Methods:**- Binding Process of Methods with Method Call Statement:**

- i. For every method call statement it has to be bind with the method call body, this process is known as **Binding**.
- ii. The binding is done by Compiler at the compile - time, hence it is known as **Compile-time binding**.
- iii. Compiler does the binding between the method call statement and the method body with the help of method signature as follows:
 1. With the help of Method Body
 2. The List of Actual and Formal Arguments must be same:
 - iv. The **Number** of Actual and Formal Arguments must be same.
 - v. The **Type** of Actual and Formal Arguments must be same.
 - vi. If it is not same there are two possibilities:
 1. Type of value will be converted to the type of Formal Argument (Widening).
 2. If Widening is not possible it throws compile time error.

Work of main() Method:

- Execution starts from main() method.
- Execution flow is controlled by main() method.
- Execution ends at main() method.

Return Statement:

- Return is a keyword and a control transfer statement.
- Return statement will stop the execution of a method, the frame gets destroyed and control is transferred back to the calling method along with the value.

NOTE:

- i. If the return type of method is void, then return statement use is optional.
 - ii. If the return type of method is anything other than void,
 - a. It means the method wants some value to calling method
 - b. Then, it is mandatory to use return statement
 - iii. Return Statement must have a value or an expression to be returned back to the calling method.
- Return statement can do widening.
 - A value return by a method can be stored inside a variable, it can be used in a expression directly or do nothing with that value.
 - If return type of method is void we can use return statement without any value.
 - Return statement must be the last statement in the method block.
 - If we have any other statement after return statement we will get "unreachable statement" Compile Time Error.

Static:

- *Static is a keyword and a modifier.*
- *Static keyword can be prefixed to a **method**, **variable** and **block**.*
- *The members which are prefixed with a static modifier are known as static members.*

NOTE:

- *Static members are considered as class members.*
- *The static members will be given with some memory in the class static area.*
- *Static members of a class can be used directly in any members of the same class, and also can be used in any member belongs to any other class with the help of class name as a reference.*
- *We can use a static member without creating any object for a class.*

Static Methods:

- *The methods which are prefixed with a static modifier are known as static method.*

Interview Questions:

- *What is Static?*
- *What is Static Member?*
- *How to call static member from one class to another?*

NOTE:

- *The method body will be stored in the method area.*
- *The method signature with its reference will be stored in class static area.*
- *A static method can be used within the same class directly with its name & also can be used in different class with class name as reference.*
- *The static method block is also known as static context.*

Dot (.) Operator:

- *It is a binary operator.*
- *It is also known as access operator.*
- ***LHS** of dot operator must be **reference**.*
- ***RHS** of dot operator must be **a member to be called**.*

Syntax:

Reference. Member to be called

LHS . RHS

Example:

`System.out.println();`

Static Variable:

- *A variable declared inside a class block prefixed with static modifier is known as static variable.*

Scope of a Static Variable:

- A static variable can be used inside any member of the same class directly and also we can use a static variable from one class to another class using class name as a reference.

NOTE:

- The memory for a static variable is given in the class static area.
- A static variable will be assigned with default value during its loading process therefore we can use a static variable without initialization.
- We can have a local variable and a static variable of same name.
- In such situation, whenever we want to use variable first local variable is referred, not the static variable.
- To use the static-variable we have to use the class name as a reference.

Static Initializers:

- Static initializers are the members which gets executed during the loading process of the class in class static area.
- That is, static initializers get executed before the execution of main method.

Use of Static Initializers:

1. To Assign the value to the static data members (variables).
2. It can also be used to perform some activities like, setting up the environment, etc. before the actual execution starts.
3. Static initializers are of two types:
 - a. **Static declare & initialization statement:**
 - Creating a static variable and storing the value simultaneously.
 - b. **Static Block.**

Static Block:

- A Block which is declared within the class block prefixed a static modifier with it, is known as Static Block.

Syntax:

```
class class_name
{
    static
    {
        statements;
    }
}
```

NOTE:

- Static Block does not have any name.
- It cannot return any value (NO RETURN TYPE)
- It does not have any access modifier.
- It does not have any formal argument.
- Programmer cannot call the static block for execution explicitly.

Que. When does a static block gets executed?

- Before the actual execution starts (before main() method).
- In a class we can have any number of static blocks.
- Static initializers execute Top to Bottom in order.
- Static Block executes before the main method.
- We can use static Scanner, it will be visible to the whole class.
- We cannot execute static initializers without main method.

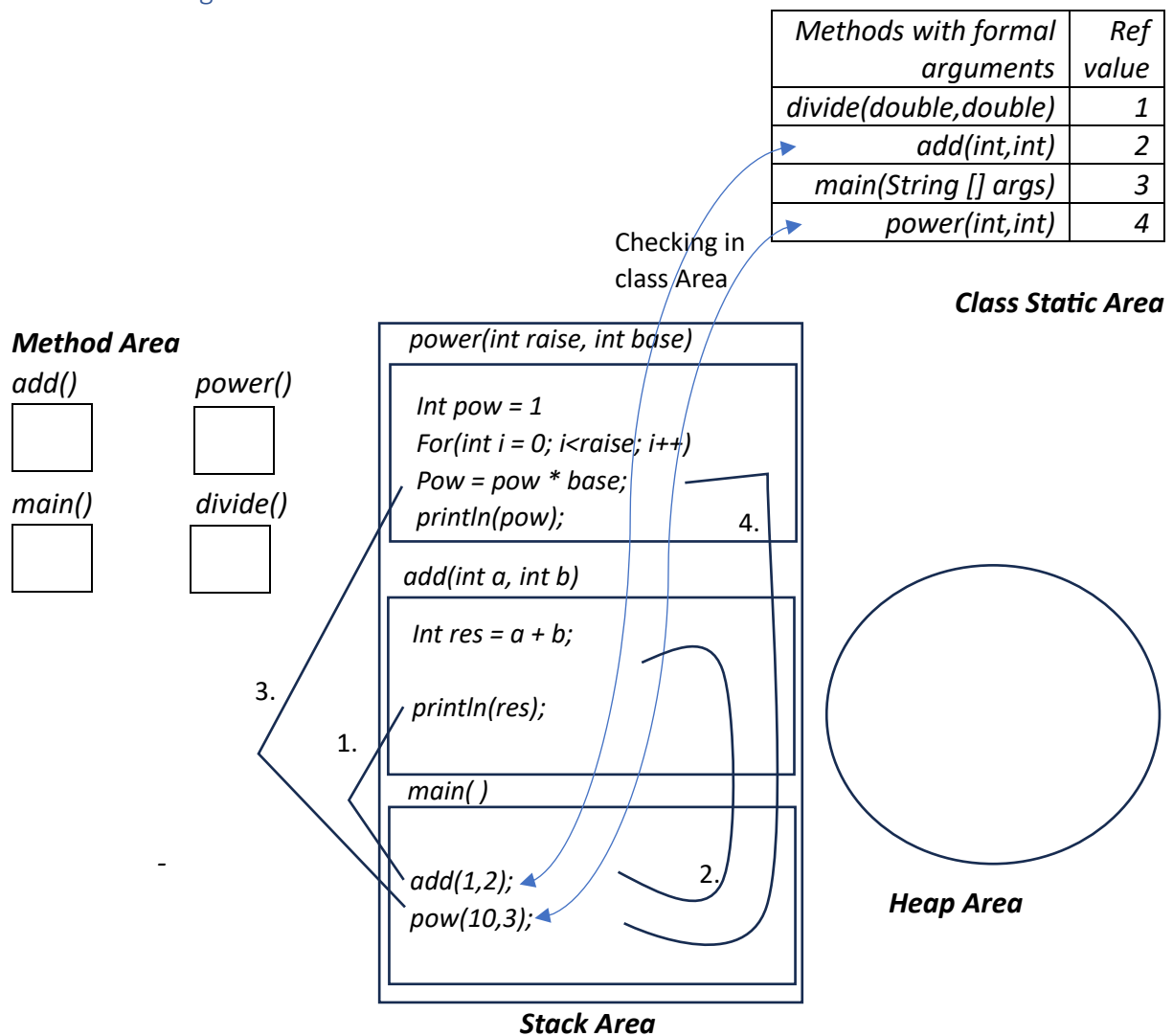
Example:

```
class StatBlock1
{
    static
    {
        System.out.println("From Static Block 1");
    }
    public static void main (String [] args)
    {
        System.out.println("From main() ");
    }
    static
    {
        System.out.println("From Static Block 2");
    }
}
```

Example:

```
import java.util.Scanner;
class StatBlock2 {
    static Scanner sc;
    static
    {
        sc = new Scanner(System.in);
    }
    public static void main (String [] args) {
        System.out.print("Enter First Number : ");
        int n1 = sc.nextInt();
        System.out.print("Enter Second Number : ");
        int n2 = sc.nextInt();
        System.out.println("Answer is : "+(n1+n2));
        merge();
    }
    public static void merge() {
        System.out.print("Enter the First Name: ");
        String fname = sc.next();
        System.out.print("Enter the Last Name: ");
        String lname = sc.next();
        System.out.print(fname+" "+lname);
    }
}
```

Method Loading & Execution Process:



Class Loading Process:

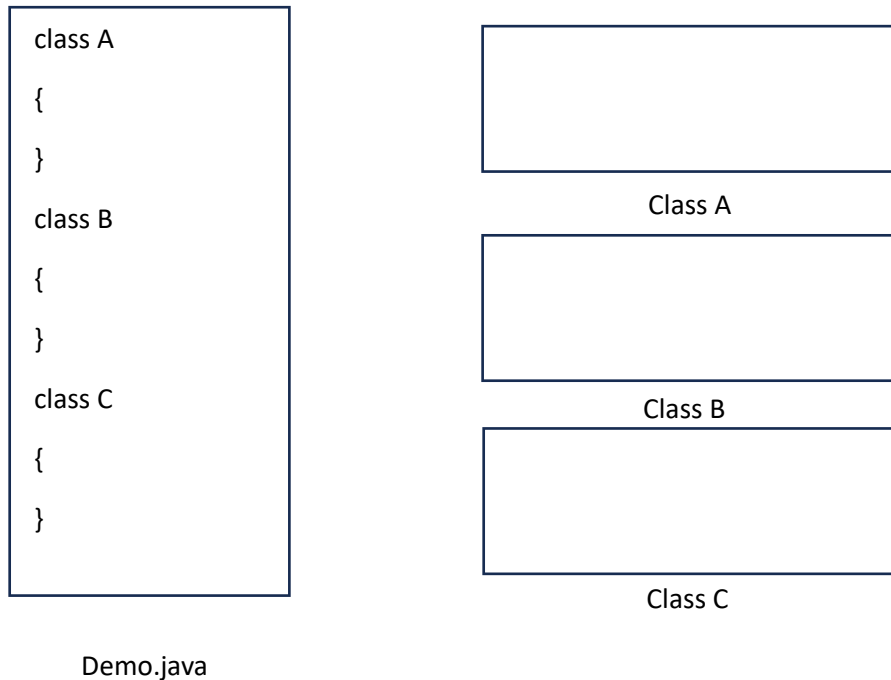
- A Memory block will be created for a class and will be divided into two different parts.
- All the method blocks will be loaded in method area and if the methods are static they are stored in the table with the reference and method signature in class static area.
- If there are any static variable, they are given with the memory in class static area and will be assigned with the default values.
- If there are any initializer, all of them will be executed. Top TO Bottom in Order.
- This completes the loading process of class.
- After this the actual execution starts from main() method.

NOTE:

- We cannot use a non-static member inside a static context directly with its name.
- We can use a static member inside a static context directly with its name.
- To use a non-static member inside a static context, we have to call its object reference.

Multiple Classes:

- If a java file contains more than one class it is known as multiple classes.
- Here, the compiler will create one class file for every class present in java file.

**NOTE:**

- In this above case the java file name is anything.

Public Class:

- A class prefix with “public” access modifier is known as public class.

RULE:

- When the class mode is public, the java file name should be same as the class name else we will get the compile time error.

NOTE:

- We cannot have more than **one** public class.
- We can achieve multiple classes where the file name must be same as the public class name.

OBJECT ORIENTED PROGRAMMING:**Que. What is Object Oriented Programming?**

- Writing or Creating programs which are related to real world entities, is known as Object Oriented Programming.

Que. What is an Object?

- Object is a real-world entity or something which has existence in the real world are known as object.

Que. Why do we need object?

- Objects help us to do day-to-day tasks easily and efficiently.

Every Object Consists of 2 Things:

1. Details / Features / Attributes / Properties:
 - This helps the programmer to identify or define the object.
2. Actions / Behaviour:
 - This specifies programmer how to use that object.

Example:**1. Mobile:**

- a. **Attributes:** Screen, Shape, Speaker, Camera, Battery, Operating System, RAM, Processor, Sensors, Brand
- b. **Actions:** Calling, Texting, Editing, Playing, Calculations, Editing, Meeting, Screening.

2. Car:

- a. **Attributes:** Colour, Model, Engine, Seats, Wheel, Brand.
- b. **Actions:** Driving, Parking, Accelerating, Braking, Turning.

3. Book:

- a. **Attributes:** Title, Author, Genre, Pages, Publisher, Year.
- b. **Actions:** Reading, Flipping Pages, Bookmarking, Closing, Opening.

Que. What is Object in Java?

- An object is a block of memory, which is created at the runtime in the Heap Area. It is a representation of a real-world entity.

NOTE:

- To create any object or before creating an object in real-world, a design or a blueprint is created.
- It provides all the specification of an object.

Example:

- Before manufacturing a car, blueprint of a car is created.
- Before constructing a house, a blueprint of a house is created.

Similarly,

- In java, before creating an object, a blueprint must be created which contain all the specification of that object.
- This design is done with the help of a class.

Class:**Que. What is Class? Define a Class?**

- A class is a user-defined non-primitive data type which represents a blueprint of an object and provides all the specifications of an object.
- Attributes or details of an object is represented by **non-static variables**.
- Actions or Behaviours are represented by **non-static methods**.

Blue-Print:

```
Class Book
{
    Statements
}
```

Driver-Class:

```
Class BookDriver
{
    Public static void main (String [] args)
    {
        Book b1 = new Book();
    }
}
```

Que. How to create an Object in Java?

- We can create an object in java with the help of “new” keyword (operator) and a constructor of a class.

“new” Keyword / Operator:

- It is a keyword and an operator which creates a block of memory in heap area at the runtime.
- new operator gets the reference of the object created and it returns it from the heap area.

Constructor:

- It is a Special Member of a class whose name is same as class name.
- A constructor loads all the non-static members of a class into the object.
- It is also known as Special Method.

Non-Primitive Data-type:

- Every class name is a non-primitive data type.
- With the help of non-primitive data-type we can create reference variable which can store the reference of an object created.

Syntax:

```
Non-primitive datatype variable_name = new class_name();
Class_name variable_name = new class_name;
```

NOTE:

- Default value of non-primitive variable (reference variable) is null.

Example:

- **Computer c1 = null;**
- **Sneakers s1 = null;**

Que. What is the difference between non-static variable and non-primitive variable?

- *Non-static variables are variables which are used into object as an attribute.*
- *Non-primitive variables are used to store the reference of non-static object.*

Non-Static Variables:

- *Variables which are declared in class block and not prefixed with static modifier is called as a non-static variable.*
- *For a non-static variable, the memory is allocated inside the object of a class.*
- *Non-Static variables are created inside every object created for a class.*
- *Non-static variables will be initialized with default values during the loading process of the object.*

Non-Static Method:

- *It is a method declared in the class block (blueprint), which is not prefixed with static modifier is known as non-static method.*
- *A non-static method definition is stored in the method area.*
- *The non-static method signature with the reference is stored inside the object of a class.*
- *A non-static method block is called as non-static context.*
- *This non-static context will hold the reference of its object, we can use the non-static member of the same class without any reference.*

Que. How to call a non-static method?

- *We can call non-static method using object as a reference.*

Que. How to call non-static method from one class to another class?

- *Yes, with the help of the object reference (by creating the object). In same class we can call it directly.*

NOTE:

- *Inside a non-static method we can call:*
 - Static members of class directly.*
 - Non-static members of class directly.*

This Keyword:

- *It is a keyword.*
- *It is a non-static reference variable.*
- *Which will have the reference of the current object.*
- *Which is used under:*
 - Under execution.*
 - Which is getting Created.*

Purpose:

- *It is used to refer the object member (non-static members) in non-static context.*

Example:

- if we have local variable and non-static variable of same name, we use **this** keyword in order to refer the non-static variable.

Constructor:

- Constructor is a **Special Member** of the class.
- It is also called as a special method whose name is same as class name and which does not have any return type.
- A programmer cannot call a constructor anywhere explicitly, but it can only be used along **"new operator"**.

Purpose of Constructor:

- It is used to load all the non-static members of the class into its object.

NOTE:

- Every class must have a constructor, it is mandatory.
- If a programmer fails to create any constructor, then at the compile-time the compiler will add a no-argument constructor into the class which is known as default constructor.

Inside a Constructor Body:

1. Instruction to load non-static members will be present.
2. All non-static initializers of the class or it will be inserted inside a constructor body.
3. User written instructions will be present inside the constructor body.

Constructors are classified into 2 types:

1. No-Argument Constructor.
2. Parameterized Constructor.

NOTE:

- Constructor added by compiler is known as default constructor.
- Constructors added by programmers are known as User-Defined Constructors.

1. No-Argument Constructor:

- Constructors without any formal arguments are known as No-Argument Constructors.

Syntax:

```
class_name() {  
    User Defined Instructions  
}
```

Example:

```
Shirt()  
{  
    System.out.println("No-Argument Constrcutor");  
}
```

2. Parameterized Constructor:

- Constructor with any formal arguments is known as Parameterized Constructors.

Purpose of Parameterized Constructor:

- It is used to assign value to the non-static variables of the object.
- A programmer can use a constructor to give startup instructions.

NOTE:

- Constructors neither be static nor non-static.
- Constructors cannot have return type.
- A default constructor will add to the blueprint only if there are no constructors in the blueprint.
- A compiler does not add the default constructor when there is already parameterized constructor is present in the blueprint.

Syntax:

```
class_name(Formal Arguments)
{
    User-Defined Instruction
}
```

Example:

```
Class Mobile
{
    String brand;
    double price;
    String colour;
    String name;
    Mobile () {
        System.out.println("From No-Argument Constructor");
    }
    Mobile(String brand, String colour, String name, double price) {
        this.brand = brand;
        this.name = name;
        this.colour = colour;
        this.price = price;
    }
}
```

Que. Order of execution of non-static members.

- Non-Static Initializers executes first but **Constructor** starts the execution first.

Non-Static Initializers:

- *Non-Static initializers are the members which execute during the loading process of the object.*
- *We can have non-static initialization in two ways:*
 - i. Non-Static Declare and Initialization Statement.*
 - ii. Non-Static Block (Instance Initialization Block).*

1. Non-Static Declare and Initialization Statement:**Syntax:**

```
Class class_name {  
    [Access Modifier] datatype variable = value / Expression;  
}
```

Purpose:

- *When all the objects created out of the class have an attribute value same. In such situation we initialize the value directly to the non-static variable in the blueprint itself.*

2. Non-Static Block:

- *A block which is declared in the class block without prefixing any modifiers is called as non-static block.*
- *A non-static does not have any name.*
- *Non-static does not have any return type.*
- *It won't accept any arguments.*
- *A programmer cannot call a non-static block explicitly for execution.*

NOTE:

- *Non-Static initializer get executed during the loading process of the object.*

Que. Order of execution of non-static members?

- 1. Constructor starts execution.*
 - 2. Non-static declaration and initialization statements.*
 - 3. Non-static variables and blocks.*
 - 4. User-defined instructions and constructor.*
 - 5. Constructor execution ends.*
- *When Constructor execution ends then the object creation is completed.*

Constructor Overloading:

- *A class having more than one constructor is known as Constructor Overloading.*

Rules:

- 1. Name of the constructor must be same.*
- 2. The formal arguments of the constructor must change, either in its type or number of formal arguments.*

Purpose:

- *We do constructor overloading when we want to customize the loading of the values to the object members.*

Method Overloading:

- If a class has more than one method of same name but with different arguments is known as Method Overloading.

Rules:

1. The name of methods must be same
2. Formal arguments must be different, either in its length or in its data-type.

Purpose:

- We do method overloading when the operations (functionality) to be performed is same. But, the values are different in its type or number's.

NOTE:

- Method Overloading comes under:
 - i. Compile Time Polymorphism (Compile-Time Binding)
- Here, the binding between the method call statement and the method body is done by the compiler at the compile-time (**refer method binding process**).

INTERVIEW QUESTIONS:

Que. Why Constructor Overloading is needed?

Que. What is Constructor Overloading?

Que. Why Method Overloading is needed?

Que. What is Method Overloading?

Tip – We can overload main() method [arguments will be different].

Method Chaining:

- The process or design of calling methods simultaneously / continuously one after another is called as Method Chaining.

Syntax:

Ref.m1().m2().m3(). ...

NOTE:

- We can call the method in this pattern only if the method returns a reference for another method to execute.

INTERVIEW QUESTIONS:

Que. What is Constructor Chaining?

Constructor Chaining:

- One constructor calling another constructor for execution is known as Constructor Chaining.
- Constructor Chaining can be achieved in 2 ways
 - i. Using this() call statement.
 - ii. Using super() call statement.

this() Call Statement:

- this() call statement is used to achieve constructor chaining within the **Same Class**.

Rules:

1. *this()* call statement can only be used inside the constructor body.
2. *this()* call statement must be the first statement in the constructor body.
3. Recursive call of the constructor is not allowed (constructor calling itself is not allowed).
4. If a class have 'n' number of constructors we can use *this()* call statement for 'n-1' times.

Example:**BluePrint:**

```
class Form
{
    String name;
    String loc;
    long phNo;

    String area;
    int houseNo;

    Form(){}

    Form(String name, String loc)
    {
        this.name = name;
        this.loc = loc;
    }
    Form(String name, String loc, long phNo, int houseNo)
    {
        this(name,loc); //Constructor Chaining
        this.phNo = phNo;
        this.houseNo = houseNo;
    }
    Form(String name, String loc, long phNo, int houseNo, String area)
    {
        this(name,loc,phNo,houseNo); //Constructor Chaining
        this.area = area;
    }
    public void printAll()
    {
        System.out.println(name);
        System.out.println(loc);
        System.out.println(phNo);
    }
}
```

```

        System.out.println(area);
        System.out.println(houseNo);
    }
}

```

Driver Class:

```

class FormDriver
{
    public static void main (String [] args)
    {
        Form f1 = new Form("Vinay", "Pune", 9356454331, 689, "Kasaba-Peth");
        f1.printAll();
    }
}

```

Packages:

- It is a mechanism of organising or grouping the similar java files.
- A group of related classes and interfaces put together to form a single entity and works for a common goal or functionality is known as **package**.

Packages are classified into 2 types:

1. **User-defined Packages.**
2. **Built-in Packages.**

Built-in Packages:

- The packages created by the developers of language (creators) which are directly available to use are known as built-in packages.

User-Defined Packages:

- The packages created by the programmer (users of the language) are known as User-Defined Packages.

Advantages:

1. Modularity of the software is increases.
2. Maintainability of the software improves.
3. Security of the software increases.
4. Code Reusability increases.

Package Creation:

- We can create packages using a keyword called 'package' followed by package name.

NOTE:

- Package creation Statement must be the first statement of the Program.

Syntax:

```

Package package_name;
Import library_files;
Class class_name {
}

```

Principles of Object-Oriented Programming:

- Principles of OOPs helps the programmer to design software easily and efficiently.

There are four principles in OOP:

- i. Encapsulation
- ii. Inheritance
- iii. Polymorphism
- iv. Abstraction

Encapsulation:

- It is a process of wrapping or binding the attributes and behaviours of an object together is known as Encapsulation.

NOTE:

- We should not give direct access to the attributes of a class (object) to the user.
- But the user should be able to read or modify the attributes of a class (object) through behaviours.
- Therefore, it is a good practice of a programmer to design a class such that we have methods to read the attributes as well as update the attributes.

Advantages:

- Since we provide indirect access to the attributes, it is secure and we can have a validation step before reading and updating the attributes.

Example:**Account Blueprint:**

```
class Account
{
    private String bankName;
    private String userName;
    private long accNo;
    private String ifsc;
    private String branch;

    private double balance;
    private int pin;

    Account()
    {
    }

    Account(String bankName, String userName, long accNo, String ifsc,
String branch,double balance,int pin)
    {
        this.bankName = bankName;
        this.userName = userName;
        this.accNo = accNo;
```

```
        this.ifsc = ifsc;
        this.branch = branch;
        this.balance = balance;
        this.pin = pin;
    }

    public String getBankName()
    {
        return bankName;
    }
    public String getUsername()
    {
        return userName;
    }
    public long getAccNo()
    {
        return accNo;
    }
    public String getIfsc()
    {
        return ifsc;
    }
    public String getBranch()
    {
        return branch;
    }

    public void withdraw(long accNo, int pin, int amt)
    {
        if (this.accNo == accNo && this.pin == pin && amt > 0)
        {
            System.out.println("Login Successful!!!");
            if (balance-amt > 0)
            {
                balance = balance - amt;
                System.out.println("Money Debited Rs."+amt);
                System.out.println("Remaining Balance :
"+balance);
            }
            else
            {
                System.out.println("Insufficient Balance !!!");
            }
        }
        else
        {

```

```

        System.out.println("Login Failed!!!");
    }
}

public void deposit(long accNo, int pin, int amt)
{
    if (this.accNo == accNo && this.pin == pin && amt > 0)
    {
        System.out.println("Login Successful!!!");
        balance = balance + amt;
        System.out.println("Money Credited: " + amt);
        System.out.println("Current Balance : "+balance);
    }
    else
    {
        System.out.println("Login Failed!!!");
    }
}

public double getBalance(long accNo, int pin)
{
    if (this.accNo == accNo && this.pin == pin)
    {
        System.out.println("Login Successful!!!");
        System.out.print("Your Available Balance is :");
        return balance;
    }
    else
    {
        System.out.println("Login Failed!!!");
        return 0;
    }
}

public void setPin(long accNo, int oldPin, int newPin)
{
    if (this.accNo == accNo && this.pin == oldPin)
    {
        System.out.println("Login Success! Please Enter New
Pin...");
        pin = newPin;
    }
    else
    {
        System.out.println("Login Failed!!!");
    }
}

```

```
        }  
    }  
  
    Account Driver:  
    class AccountDriver  
    {  
        public static void main (String [] args)  
        {  
            Account a1 = new Account  
("SBI","Vinay",123456789I,"SBI123","Pune",100000.0,1234);  
            a1.withdraw(123456789I,1234,50000);  
        }  
    }
```

INTERVIEW QUESTIONS:

Que. What is Data Hiding?

Que. What is Encapsulation?

Que. How to Achieve Data Hiding?

Data Hiding:

- *It is a process of restricting the direct access to the attributes of a class (object) and providing the indirect and secured access to the properties of a class (object) is known as Data Hiding.*

Steps to achieve Data Hiding:

- Make the **attributes private**.*
- Design **getter & setter** methods for **reading & updating** the attribute **respectively**, this helps the programmer to get flexibility to design the following:*
 - *We can make an attribute **readable** but not modifiable*
 - *We can make an attribute **modifiable** but not readable.*
 - *We can make an attribute both **readable & modifiable**.*
 - *We can make an attribute **neither** readable **nor** modifiable.*

Relationship:

- Association between two or more objects is known as Relationship.
- It is classified into 2 types
 - i. Has-A-Relationship
 - ii. Is-A-Relationship

OBJECT 1:

```

Class Battery {
    Brand;
    Mah;
    Price;
    Battery () {
        }
    Public void displayBattery () {
        }
    }

```

OBJECT 2:

```

Class Mobile {
    Model;
    mobBrand;
    noOfCam;
    price;
    Battery b1 = new Battery();
    Mobile () {
        }
    Public void displayMobile () {
        }
    }

```

1. Has-A-Relationship:

- It is association between two or more objects where one object is dependent on another object.
- Based on level of dependency Has-A-Relationship is categorized into 2 types
 - i. Composition
 - ii. Aggregation

Que. How to achieve Has-A-Relationship?

- An object must have reference of another object
- We can easily achieve this design by creating reference variable of one object inside another class.
- By considering the above design we have achieve Has-A-Relationship where the mobile object will have the reference of the battery object.

- We can achieve either composition or aggregation by creating reference of one object (same object) in another object (main object).
- When the service user of the mobile can create an object of mobile, automatically object of battery gets created and its reference gets stored in the mobile object.
- We can achieve Has-A-Relationship in 2 ways:
 - i. Early Instantiation (Early Binding)
 - ii. Lazy Instantiation (Lazy Binding)

1. Early Instantiation (Early Binding):

- In early instantiation the instance of dependent object is implicitly created under the instance of object created.
- The design is achieved with the non-static initializers or constructors.

NullPointerException:

- We get Null pointer Exception when we try to invoke the member using a reference variable storing with null.
- Exception will come at runtime.

2. Lazy Instantiation (Lazy Binding):

- In lazy binding the instance of a dependent object is not created implicitly with the help of non-static initializer instead a method is created and the reference gets created in main object when method is called for execution, this method is called as Helper Method.

Program:

Pen drive Class:

```
package LazyBinding;
public class Pendrive {
    String brandName;
    double storage;
    double price;
    double speed;
    public Pendrive(String brandName, double storage, double price,
double speed) {
        super();
        this.brandName = brandName;
        this.storage = storage;
        this.price = price;
        this.speed = speed;
    }
    public void displayPendrive() {
        System.out.println("Brand : "+brandName);
        System.out.println("Storage : "+storage);
        System.out.println("Price : "+price);
    }
}
```



```

        System.out.println("Speed : "+speed);
    }
}

```

Computer Class:

```

package LazyBinding;
public class Computer {
    String brandName;
    String os;
    int ram;
    int hdrive;
    String processor;
    Pendrive p;
    public Computer(String brandName, String os, int ram, int hdrive,
String processor) {
        super();
        this.brandName = brandName;
        this.os = os;
        this.ram = ram;
        this.hdrive = hdrive;
        this.processor = processor;
    }

    public void insertPendrive(String brandName, double storage, double
price, double speed) {
        p = new Pendrive(brandName, storage, price, speed);
        System.out.println("Pendrive Inserted!");
    }

    public void displayComputer() {
        System.out.println("Brand : "+brandName);
        System.out.println("Operating System : "+os);
        System.out.println("RAM : "+ram);
        System.out.println("Hard Drive : "+hdrive);
        System.out.println("Processor : "+processor);
    }
}

```

Driver Class:

```

package LazyBinding;
public class ComputerDriver {
    public static void main (String [] args) {
        Computer c1 = new Computer("Lenovo", "Windows", 16, 512,
"Ryzen-7 5800H");
        c1.insertPendrive("SanDisk", 256, 1500, 500);
    }
}

```

```
        c1.displayComputer();  
        System.out.println();  
        c1.p.displayPendrive();  
    }  
}
```

Output:

Pendrive Inserted!
Brand : Lenovo
Operating System : Windows
RAM : 16
Hard Drive : 512
Processor : Ryzen-7 5800H
Brand : SanDisk
Storage : 256.0
Price : 1500.0
Speed : 500.0

2. Is-A-Relationship:

- The relationship between two or more objects, which is similar to parent & child relation is known as Is-A-Relationship.
- In Is-A-Relationship the child object requires all the properties and behaviours of the parent object along with its attributes and behaviours.
- With the help of Is-A-Relationship we can achieve generalization and Specialization.

NOTE:

- Here, parent class is known as Generalized & child class is known as Specialized.
- With the help of **child class object reference** we can use all the members of parent class as well as child class.
- With the help of **parent class object reference** we can use all the members of parent class but not child class Members.

Terminologies:**1. Parent Class:**

- Parent class is also known as **Super class** or **Base Class**.

2. Child Class:

- Child class is also known as **Sub Class** or **Derived Class**.
- we can achieve Is-A-Relationship with the help of Inheritance.

Inheritance:

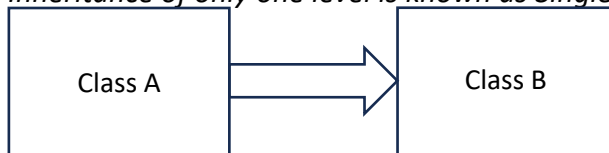
- It is a process of one class acquiring all the properties and behaviours from another class is known as Inheritance.
- In Java, we can achieve inheritance using
 - i. "extends" keyword
 - ii. "implements" keyword

Levels of Inheritance:

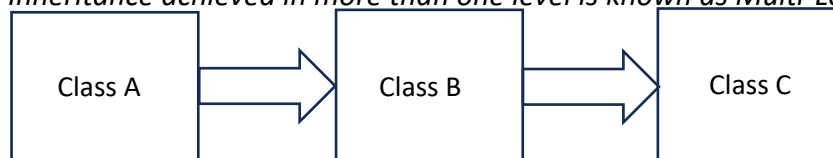
1. Single Level Inheritance
2. Multi-Level Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

1. Single Level Inheritance:

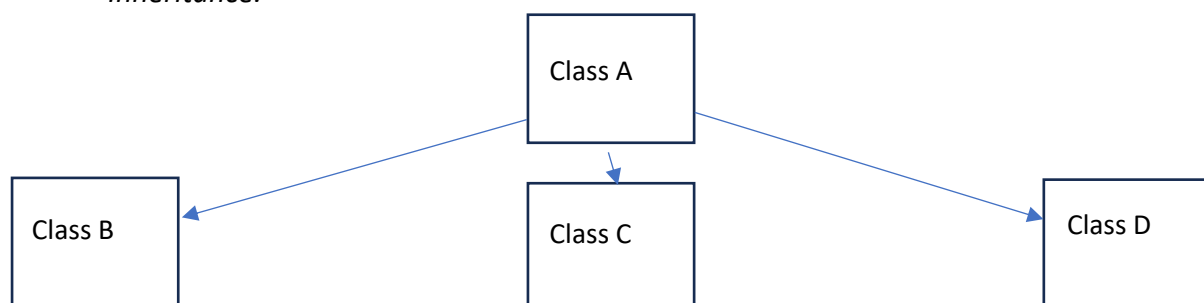
- Inheritance of only one level is known as Single Level Inheritance.

**2. Multi-Level Inheritance:**

- Inheritance achieved in more than one level is known as Multi-Level Inheritance.

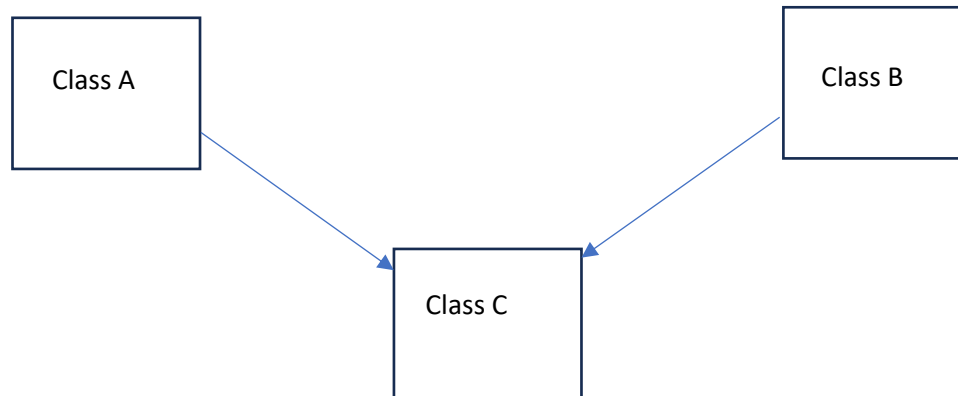
**3. Hierarchical Inheritance:**

- A Superclass having more than one subclass on same level is known as hierarchical Inheritance.



4. Multiple Inheritance:

- A sub class derived from more than one super class of same level is known as Multiple Inheritance.



- Multiple Inheritance has an ambiguity (confusion) known as Diamond Problem.

Que. What is Diamond Problem?

- Diamond Problem is an ambiguity (confusion) which occurs to the subclass object, when we try to invoke the members of super class.
- EXAMPLE:
 - Let us consider the above scenario, i.e class Virat and class Anushka having a common child class called Aakay.
 - If class Virat and class Anushka are having same method signature (profession) , then there is a confusion whether to inherit it from class Virat or from class Anushka.
 - This confusion is termed as 'Diamond Problem'.
 - Therefore, creators of Java, instead of resolving this issue, they restricted inheritance of a class with only one class.
 - Therefore, in Java, class can extends to multiple inheritance with respect to classes, but this design is available in java using the concept known as **Interfaces**.
- These ambiguities occur because of
 - Case 1:**
 - When both the parent classes have same method signature.
 - Case 2:**
 - If a `super()` call statement is used in subclass constructor, there is an ambiguity for the `super()` call statement whether to invoke the constructor of first parent or second parent.

INTERVIEW QUESTIONS:

Que. How to pass the values from subclass to super class?

Que. How to do constructor chaining within same class as well as another class?

Que. Why we need super call statement?

Super () Call Statement:

- *Super call statement is used to call the super class constructor during the creation of subclass object.*
- *It helps us to load the non-static members of the super class in the subclass object.*

NOTE:

- *Super() call statement must be the first statement inside a constructor body.*
- *Super() call statement can be only be used inside a constructor.*
- *If a programmer does not add a super() call statement implicitly, the compiler will add a no argument super call statement.*
- *A constructor cannot have both this() and super() call statements.*
- *If this() call statement is used in a constructor then the compiler will not add the default super() call statement.*

Purpose:

- *It helps us to load the non-static members of the super class into the subclass object.*
- *We can pass the data from subclass to superclass, using super() call statement.*

NOTE:

- *A class can inherit another class with the “**extends**” keyword.*
- *Both **static** and **non-static members** of class get **inherited**.*
- *With the help of subclass object reference, we can access both static and non-static members of both subclass and superclass.*
- *Private members of a class cannot be inherited.*
- *Constructors of a class cannot be inherited.*
- *Initializers (Static Block, non-static Block) of a class cannot be inherited.*
- *To use private members of parent class we have to use **getter & setter** methods.*

Non-primitive Typecasting / Derived Typecasting:

- It is a process of converting or storing one type of non-primitive data (object reference) into another type.

NOTE:

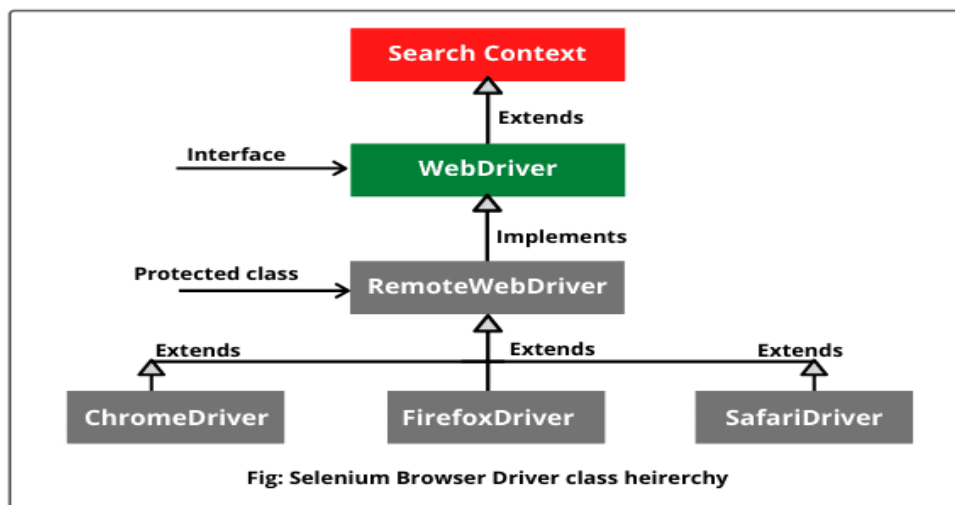
- We can perform derived typecasting only if two classes (parent & child) having Is-A-Relationship.
- Derived typecasting is classified into 2 types:
 - i. Upcasting
 - ii. Down-casting

1. Upcasting:

- Upcasting is a process of converting or storing subclass type into superclass type.
- When the subclass reference is upcast into superclass reference, we cannot use the members of subclass with super class (upcasted) reference.
- Upcasting is done implicitly by the compiler.

Purpose of Upcasting:

- A purpose of upcasting is to achieve generalization. (any child object upcasted to parent type can show the properties and behaviours of parent object but not itself).



By considering the above hierarchy,

Case 1:

- If we create the reference variable of **ChromeDriver**, we can only store a reference of instance of **ChromeDriver**.

Case 2:

- If we create the reference variable of **FirefoxDriver**, we can only store a reference of instance of **FirefoxDriver**.

Case 3:

- If we create the reference variable of *SafariDriver*, we can only store a reference of instance of *SafariDriver*.
- **But**, If we create a reference variable of super class (*WebDriver*), then because of upcasting we can store a reference of instance of *ChromeDriver*, *FirefoxDriver* and *SafariDriver* as well.

2. Down-casting:

- It is a process of converting or storing the **upcasted reference** back to respective subclass type.

Purpose:

- If a reference is upcasted, we cannot use the members of subclass. Therefore, to use the members of subclass, we have to perform down-casting.

NOTE:

- Down-casting is not done implicitly by the compiler.
- It should be done explicitly by the programmer with the help of typecast operator.

Syntax:

subclassType var = (subclassType) upcasted ref;

Example:

Parent Class:

```
class Father
{
    public void drinking()
    {
        System.out.println("Drinking");
    }
}
```

Child Class:

```
class Son extends Father
{
    public void smoking()
    {
        System.out.println("Smoking");
    }
}
```

Driver Class:

```
class FatherSonDriver
{
    public static void main (String [] args)
    {
        Father f1 = new Son(); //Upcasting
    }
}
```

```

        f1.drinking();

        Son s1 = (Son) f1; //Down-Casting
        s1.smoking();
    }
}

```

INTERVIEW QUESTIONS:**Que. Difference between “this” & “super” keyword?****Que. Difference between this() & super() call statement?****ClassCastException:**

- When we try to downcast the reference to a specific subclass type but, the object does not contain attributes or behaviours of subclass then we get ClassCastException.

OR

- When we try to Downcast without Upcasting or which is not upcasted, we get ClassCastException.

Super Keyword:

- Super Keyword is used to refer the attributes and behaviours of the super class members in the subclass, when both superclass & subclass are having attributes and behaviours of same name.

instanceOf:

- instanceof is a binary operator, which is used to check whether the given object reference (object) contains the members of the specified class or not.
- The L.H.S must be object reference and R.H.S must be class name.

Syntax:**Ref instanceof ClassName;****Example:**

```

class InstanceOf
{
    public static void main (String [] args)
    {
        //Y Extends X
        Y y1 = new Y();
        System.out.println(y1 instanceof Y); //true
        System.out.println(y1 instanceof X); //true
        X x1 = new X();
        System.out.println(x1 instanceof X); //true
        System.out.println(x1 instanceof Y); //False
        System.out.println(x1 instanceof Father); //CTE
    }
}

```


Polymorphism:

- **Poly** means many, **morph** means form.
- It is an ability of a java member to have one name and many forms.
- It is of 2 types:
 - i. **Compile time polymorphism.**
 - ii. **Runtime polymorphism.**

1. Compile-time Polymorphism:

- If a member (variable or methods) has one name and many forms, here the compiler decides which forms to be executed at compile time, this is known as Compile-time Polymorphism.
- Compile-time polymorphism achieve with:
 - i. Method Overloading
 - ii. Constructor Overloading
 - iii. Method Shadowing
 - iv. Variable Shadowing
 - v. Operator Overloading (not supported except "+" operator for addition & Concatenation)

2. Runtime Polymorphism:

- If a member (method) has one name and many forms, here compiler will not do the decision, but the member is given to the preference on the basis of type of object created.

Que. How to achieve Runtime Polymorphism?

- Derived Typecasting (Upcasting)
- Method Overriding.

Method Overriding?

- It is a mechanism of giving new implementation (design) to a superclass method from the subclass is known as Method Overriding.

NOTE:

- Method Overriding can be done only with non-static methods.
- In order to do method overriding, the classes must be in **Is-A-Relationship**.
- We have to design exact same method in subclass with same implementation (body).
- We have to create a subclass object and upcast it to respective superclass type.
- When we upcast according to rule of upcasting, the binding will be done by considering the type of reference used to invoke the method by the compiler at the compile-time.
- At runtime, JVM changes the decision by giving priority to the method which belongs in type of object created, but not to the type of reference used to invoke the method, this is known as **Method Overriding**.

Method Shadowing:

- If we have **static methods** of same declaration in both super class and subclass, it is known as Method Shadowing.
- Method Shadowing is compile-time polymorphism. That is, execution of method definition (body) does not depend on the type of object created, instead it depends on type of reference use to invoke the method.

NOTE:

- Static Methods cannot be overridden, if we try to do it is Method Shadowing.

Variable Shadowing:

- If a superclass and subclass have **static or non-static** variables of same name, then it is known as Variable Shadowing.
- It is compiler time polymorphism.
- The execution of method definition does not depend on the type of object created, instead it depends on type of reference use to invoke the method.

Abstraction:

- *Abstraction is a design process which helps the service specifier to specify the most essential features of a project or a software without providing their implementation details.*

OR

- *In process of hiding implementation details and only showing essential details is known as Abstraction.*

OR

- *Features with implementation details.*

Que. How to achieve Abstraction?

- *With the help of classes.*
- *With the help of interfaces.*

Methods are classified into 2 types:

- Abstract Methods*
- Concrete Methods*

1. Abstract Methods:

- *Methods without any implementation details (body) are prefixed with “**abstract**” keyword / modifier are known as Abstract Methods.*

Syntax:

AccessModifier abstract returnType methodName (Formal Arguments)

NOTE:

- *If a class has at-least one abstract method declared or inherited, we have to make that class abstract otherwise we get compile-time error.*

2. Concrete Methods:

- *Methods which have implementation details (body) is called as Concrete Methods.*

Classes are classified into 2 types:

- Abstract Class**
- Concrete Class**

1. Abstract Class:

- *A class which is prefixed with abstract modifier is known as Abstract Class.*
- *It is mandatory to make a class abstract, if that class having at least one abstract method either declared or inherited but **not overridden**.*

NOTE:

- *We cannot create object of an abstract class but we can create objects from a Concrete Class.*

2. Concrete Class:

- *A class which is not prefixed with abstract modifier is called as Concrete Class.*

Example:**Abstraction Class:**

```
abstract class AbstractCalculator
{
```

```
public abstract void add(int a, int b);

public abstract void subtract(int a, int b);

abstract public void divide(double num, double den);

public void greetings()
{
    System.out.println("Welcome to the Calculator");
}
}
```

Blueprint Class:

```
class Calculator extends AbstractCalculator
{
    public void add(int a, int b)
    {
        System.out.println(a+b);
    }
    public void subtract(int a, int b)
    {
        System.out.println(a-b);
    }
    public void divide(double a, double b)
    {
        System.out.println(a/b);
    }

    public void message()
    {
        System.out.println("This is Dirty Company");
    }
}
```

Driver Class:

```
class CalculatorDriver
{
    public static void main (String [] args)
    {
        AbstractCalculator c1 = new Calculator();
        c1.add(2,3);
        c1.message();
    }
}
```

Object Class:

- Object class is known as Super most class on any class in Java.
- Object class is present on "java.lang" package.
- Object class has eleven non-static methods.
- These are:
 - i. *public String toString()*
 - ii. *public boolean equals (object o)*
 - iii. *public int hashCode()*
 - iv. *public final void wait () throws InterruptedException*
 - v. *public final void wait (long millis) throws InterruptedException*
 - vi. *public final void wait (long millis, int nanos) throws InterruptedException*
 - vii. *public final void notify() throws InterruptedException*
 - viii. *public final void notifyAll() throws InterruptedException*
 - ix. *public final class getClass()*
 - x. *protected void finalize() → Deprecated*
 - xi. *protected void clone() throws NotSupportedException*

i) public String toString():

- *toString()* method of object class returns a String in the below given format:
<Fully Qualified ClassName @ Hexadecimal Number>
(Object Reference Format)
- We override *toString()* method of Object Class in order to customize the object reference format.

Example:

Bag Blueprint:

```
package ObjectClass;
public class Bag {
    String brand;
    double price;
    double cap;
    String color;

    public Bag() {
        super();
    }
    Bag(String brand, double price, double cap, String color) {
        super();
        this.brand = brand;
        this.price = price;
        this.cap = cap;
        this.color = color;
    }
}
```

```

        public String toString() {
            return "["+brand+", Price: "+price+", Capacity: "+cap+",
Color:"+color+"]";
        }
    }
}

```

Bag Driver:

```

package ObjectClass
public class BagDriver {
    public static void main (String [] args) {
        Bag b1 = new Bag("Wild Craft", 1000.0,35.0,"Blue");
        System.out.println(b1);
        System.out.println(b1.toString());
        Bag b2 = new Bag("Wild Craft", 1000.0,35.0,"Blue");
        System.out.println(b2);
        System.out.println(b2.toString());
        System.out.println(b1==b2);           //false
    }
}

```

ii) public Boolean equals (object o):

- Object Class implementation of equals() method is used to compare the reference of two objects.
- Equality Operator (==) is used to compare the reference of two objects and not their attributes.
- Object class equals() method behave exactly similar to equality operator, which compare the reference of two objects but not their attributes.
- We override equals() method, when we want to compare two objects by their attributes, but not their references.

iii) public int hashCode():

- hashCode() method of object class returns a unique number for every object created.
- We override hashCode() method, whenever equals() method compares two objects by their attributes and return **true**, hashCode() method has to generate same hash code value for both the objects, This is known as HandShake Property.

Example:**Bag Blueprint:**

```

class Bag
{
    String brand;
    double price;
    double cap;
    String color;
    char grade;
}

```

```

    public Bag()
    {
        super();
    }
    Bag(String brand, double price, double cap, String color, char grade)
    {
        super();
        this.brand = brand;
        this.price = price;
        this.cap = cap;
        this.color = color;
        this.grade = grade;
    }
    public String toString()
    {
        return "["+brand+", Price: "+price+", Capacity: "+cap+",
Color:"+color+", Grade:"+grade+"]";
    }
    public boolean equals(Object o)
    {
        Bag b = (Bag) o;
        if (this.brand == b.brand && this.price == b.price && this.cap == b.cap
&& this.color == b.color && this.grade == b.grade)
        {
            return true;
        }
        return false;
    }

    public int hashCode()
    {
        int hc = 0;
        hc = hc + price + cap + grade + brand.hashCode() + color.hashCode();
        return hc;
    }
}

```

Bag Driver:

```

class BagDriver {
    public static void main (String [] args) {
        Bag b1 = new Bag("Wild Craft", 1000.0,35.0,"Blue",'A');
        System.out.println(b1);
        System.out.println(b1.toString())
    }
}

```

```

        Bag b2 = new Bag("Wild Craft", 1000.0, 35.0, "Blue", 'A');
        System.out.println(b2);
        System.out.println(b2.toString());
        System.out.println(b1==b2);
        System.out.println(b1.equals(b2));
        System.out.println(b1.hashCode());
        System.out.println(b2.hashCode());

```

```

    }
}

```

Method Overloading in Object Class:

Blueprint Class:

```

package ObjectClass;
public class Student{
    String name;
    String college;
    int id;
    int age;
    char sec;
    char gen;
    String stream;
    public Student(){

    }
    public Student(String name, String college, int id, int age, char sec, char gen, String
stream) {
        this.name = name;
        this.college = college;
        this.id = id;
        this.age = age;
        this.sec = sec;
        this.gen = gen;
        this.stream = stream;
    }

    public String toString() {
        return "[ Name:"+name+" ,ID:"+id+" Age:"+age+" Section:"+sec+"
Gender:"+gen+
" Stream:"+stream+"]";
    }

    public boolean equals (Object o) {

```



```
Student s = (Student) o;  
return this.name == s.name && this.id == s.id && this.age == s.age &&  
this.gen == s.gen && this.sec == s.sec && this.stream == s.stream ;  
}  
}
```

Driver Class:

```
package ObjectClass;  
  
public class StudentDriver {  
    public static void main (String [] args) {  
        Student s1 = new Student("Vinay", "FTC", 123, 789, 'A', 'M', "CS");  
        Student s2 = new Student("Vijay", "FTC", 123, 789, 'A', 'M', "CS");  
        System.out.println(s1);  
        System.out.println(s1 == s2);  
        System.out.println(s1.equals(s2));  
        System.out.println(s1.hashCode());  
        System.out.println(s2.hashCode());  
        System.out.println(s1.toString());  
        System.out.println(s2.toString());  
    }  
}
```

Interfaces:

- Interface is a member in java which is used to achieve **Pure Abstraction**.
- Pure Abstraction means concrete, non-static methods are **not allowed** in Interface.
- Interfaces are used to achieve Multiple Inheritance.

NOTE:

- We can create public non-static abstract methods.
- We can create static public final variable.
- All the members declared in interface are by default public or is not necessary by programmer to use prefix public access modifier.
- All non-static methods are by default Public and Abstract, is not necessary.
- For programmer to use abstract and final modifier.
- All variables in an interface are by default public static and Final. So, it is not necessary to programmer prefix public static and final.
- In interface which contains **only one abstract method** is known as **Functional Interface**.
- If an interface does **not contain any abstract methods** is known as **Market Interface**.
- Since, JDK 1.8 we can also create public static methods is an Interface.
- We can also create default non-static methods in an interface.

1. Interface Class:

```
package Interface;
interface Calculator {
    double pi = 22.0 / 7.0;
    public abstract void add(int a, int b) {
        void div (double a, double b);
    }
}
```

2. Blue-Print Class:

```
package Interface;
public class CalculatorImp implements Calculator{
    String name;
    String brand;
    double price;
    public CalculatorImp() {
    }
    CalculatorImp(String name, String brand, double price){
        this.name = name;
        this.brand = brand;
        this.price = price;
    }
    public void add(int a, int b) {
        System.out.println(a+b);
    }
}
```

```

    }
    public void div(double a, double b) {
        System.out.println(a/b);
    }
}

```

3. Driver Class:

package Interface;

```

public class CalDriver {
    public static void main (String [] args) {
        CalculatorImp c1 = new CalculatorImp("ES", "Casio", 1000);
        c1.add(25, 25); //50
        System.out.println(Calculator.pi); // 3.142857142857143
    }
}

```

NOTE:

- We cannot have concrete non-static method in an method.
 - i. Interface cannot have any Constructor.
 - ii. We cannot create any object for an interface. That is, Interface cannot be instantiated.

NOTE:

- If we want to achieve inheritance between two class we use **extends** keyword.
- If we want to achieve inheritance between to interface we use **extends** keyword.
- If we want to achieve inheritance between class and inheritance we use **implements** keyword.
- Interface cannot inherit class.
- Concrete method get inherited on interface which is not allowed.

Multiple Inheritance is Possible in Inheritance Because,

1. Non-static methods are abstract and they don't have definition hence there is no ambiguity.
2. Static method of interfaces are any how not inherited. Hence, there is no ambiguity (To access static member with class call it with interface).
3. Interfaces do not have any constructor. Hence, there is no confusion in super call statement.
4. Static variable of interfaces are inherited to subclass only with their names are error, so it is better to call them with interface name.
5. From all this point, we can conclude there is no possibility of diamond problem. Hence, Interface support multiple Inheritance.

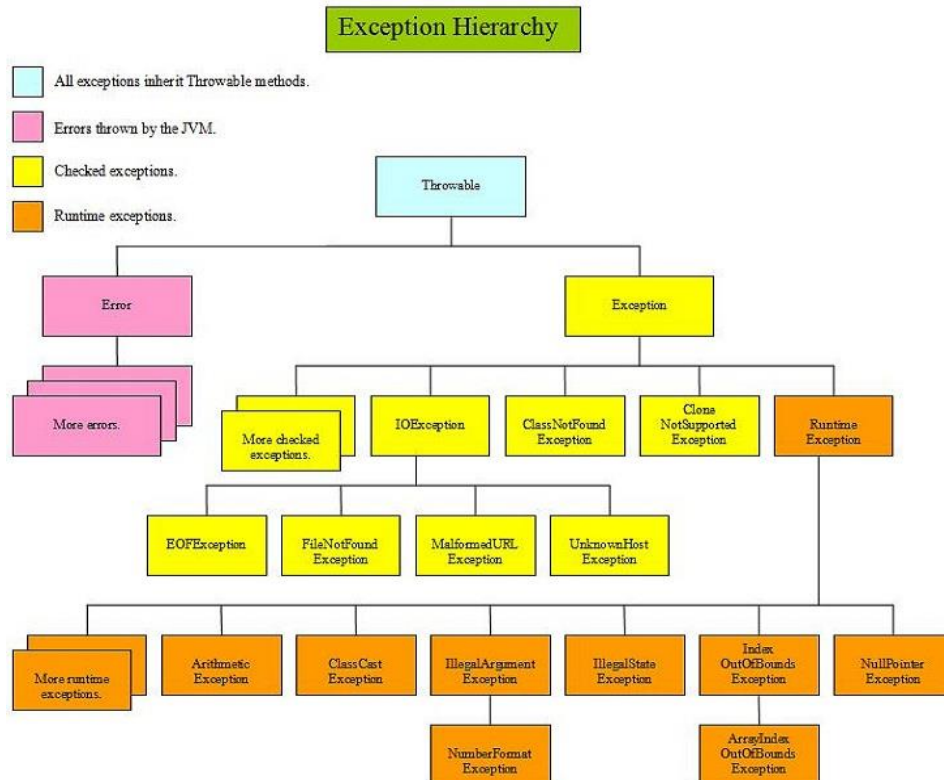
Exception & Exception Handling:

Exception:

- An exception is a problem that occurs during the execution of software due to an abnormal situation.

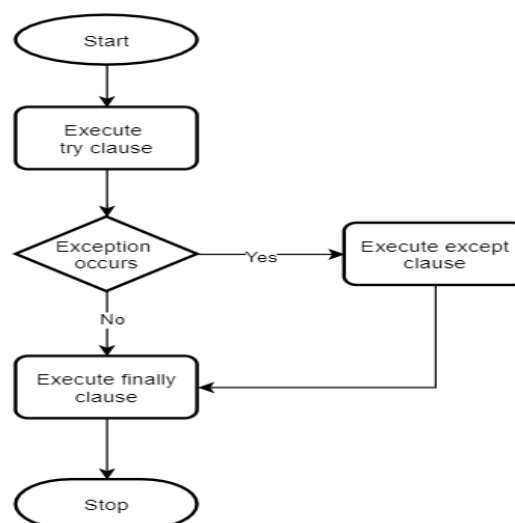
Exceptions are classified in 2 Types:

1. Checked Exception.
2. Unchecked Exception.



- *RuntimeException & their subclasses come under unchecked Exception, Rest others under Unchecked Exception.*

Exception Flow Chart:



Checked Exception:

1. *Compiler known Exceptions are called as Checked Exception.*
2. *Here the compiler knows the statements which are responsible for a particular Exception.*
3. *The compiler forces the programmer to either declare or handle the Exception.*
4. *If the execution is neither handled nor declared, the compiler throws unsupported Exception CTE.*

Unchecked Exception:

1. *Compiler unaware Exceptions are Called as Unchecked Exception.*
2. *Here the compiler doesn't know the statement which is responsible for particular Exception.*

NOTE:

- *The object class method toString() which is overridden throwable class & subclass in such way that they print the name of Exception along with the message*

Exception Handling:

- *Exception Handling is a process which has to be followed to bring back a software to normal execution which is basic & abnormal situation (Exception).*

OR

- *It is a mechanism which is used to continue the flow of execution when Exception has occurred.*
- *In Java, we can handle Exception using try() – catch() block.*

Syntax:

```
try{  
    Declare Statements which are responsible for a particular Execution.  
}  
Catch (throwable type of reference) {  


- Reason for particular Exception
- Debugging mechanism
- Step to follow to come to this situation

  
}
```

Try Block:

- *When an exception has occurred a throwable type object is created.*
- *The reference of throwable object created will throw to catch block.*

Catch Block:

- *Catch block must catch the reference thrown by try. Only then we can say Exception handled successfully.*
- *If the catch does not catch the exception reference it means the exception is not handled.*

NOTE:

- *Catch block gets executed only when Exception handled.*
- *The catch block can catch the exception thrown, only if –*
 - i. *Variable declared in catch block must be same as type of Exception Created.*
 - ii. *Variable declared in catch block can be of parent type(super class type) when exception is thrown.*

Example:

```
package Interface;
import java.util.Scanner;
public class Calculation {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter the First Number:");
        double n1 = sc.nextInt();
        System.out.print("Enter the Second Number:");
        double n2 = sc.nextInt();
        try {
            double res = n1/n2;
            System.out.println(res);
        }
        catch(ArithmeticException ae) {
            System.out.println("Please don't divide the number by zero!");
        }
        System.out.println("Thank You!");
    }
}
```

Program 2:

```
package Exceptions;
public class Exception2 {
    public static void main (String [] args) {
        try {
            String s = null;
            System.out.println(s.toString());
        }
        catch(NullPointerException np) {
            System.out.println("Don't invoke the non-static members
                               with null values!");
        }
        System.out.println("End");
    }
}
```

Try with Multiple catch () Block:

- When we have more than one statement which are causing Exception in a program then we use try with multiple catch () Blocks.
- A try block having more than one catch block is known as try with multiple catch block.

Syntax:

```
try {
    Statement 1;
    Statement 2;
    Statement 3;
}
catch (AE){
}
Catch (type) {
}
Catch (INC) {
}
```

- The reference of throwable type object is known to first catch block, If it is not caught by the reference then it goes to second catch block.
- In this way, the reference of object is given to catch block in top to bottom order until it is caught.
- If the reference is not catch by any catch-block then program goes to unplanned stop and we can say the Exception is not handled.

NOTE:

- In try block at most one Exception can be created.
- And only one catch block will be Executed.

Example:

```
package Exceptions;
import java.util.InputMismatchException;
import java.util.Scanner;

public class Exception4 {
    public static void main (String [] args) {
        try {
            Scanner sc = new Scanner (System.in);
            System.out.println("Enter Numerator:");
            int n1 = sc.nextInt();
            System.out.println("Enter Denominator:");
            int n2 = sc.nextInt();
            System.out.println(n1/n2);
        }
        catch (ArithmeticException ae) {
```

```

        System.out.println("Don't Divide by Zero!");
    }
    catch (InputMismatchException ime) {
        System.out.println("Please Enter Valid Input!");
    }
    catch (NullPointerException npe) {
        System.out.println("Don't call non-static method
        with null reference!");
    }
    System.out.println("End");
}
}

```

Order of –

Declaration of Exception Variable in catch Block:

```

try {
    Statement 1;
    Statement 2;
    Statement 3;
}
catch (AE) {
}
catch (NPE) {
}
catch (Exception) {
}
catch (throwable) {
}

```

Right Way of order of declaring Exception

```

try {
    Statement 1;
    Statement 2;
    Statement 3;
}
catch (Exception) {
}
catch (AE) {
}
catch (NPE) {
}
catch (IME) {
}

```

Wrong way of declaring Exception
// CTE unreachable catch block.

- The exception variable in catch block must be declared from child type of parent type from top to bottom order.

Finally Block:

- *"Finally Block" is used along with try or try – catch block. The control will go to finally block in the following situations –*
 - i. *The exception has occurred and handled.*
 - ii. *When an exception has occurred but not handled.*
 - iii. *Exception occur in try block & catch block is not used / declared, then the finally block gets executed & the program gets terminated.*

Que. When will Finally Block not execute?

- *When an exception occurs outside the try block, then Finally block will not execute.*

Example:

```
package Exceptions;
```

```
public class FinallyBlock {
    public static void main (String [] args) {
        try {
            System.out.println(10/0);
        }
        finally{
            System.out.println("From Finally Block");
        }
    }
}
```

Example:

```
package Exceptions;
```

```
public class FinallyBlock2 {
    public static void main (String [] args) {
        try {
            System.out.println(10/0);
        }
        catch (ArithmeticException ae) {
            System.out.println("Don't Divide by Zero!!!");
        }
        finally {
            System.out.println("From Finally");
        }
    }
}
```

Exception Object Propagation:

- It is a process of movement of execution object reference from called method to the calling method, when the exception is not handled. It is known as Exception object propagation.

Example:

```
package Exceptions;

public class ExceptionPropagation {
    public static void main (String [] args) {
        try {
            divide(10,0);
        }
        catch(ArithmeticException ae) {
            System.out.println("Don't Divide by Zero!!!");
        }
    }
    public static void divide(int a, int b) {
        System.out.println(a/b);
    }
}
```

Declaring of Exception:**Example:**

```
package Exceptions;

public class Checked {
    public static void main (String [] args) throws InterruptedException{
        for(int i = 0; i < 5; i++) {
            System.out.println("I Love India!!!");
            Thread.sleep(1000);
        }
    }
}
```

Wrapper Class:

- *Wrapper Classes are built-in classes in Java which are used to convert primitive type data into non-primitive.*
- *The process of converting primitive type data into non-primitive is known as **Boxing**.*
- *The process of converting non-primitive type data back to its primitive form is known as **Unboxing**.*

NOTE:

- *All the wrapper classes override following methods of Object Class –*
 - toString ()*
 - equals (Object o)*
 - hashCode ()*
- *All the wrapper classes implement serializable & Comparable type.*

Example:**Boxing:**

```
package Wrapper;
```

```
public class IntWrap {
    public static void main (String [] args) {
        int a = 10;
        Integer i1 = new Integer(a);
        System.out.println("Non-Primitive int:"+i1);
        System.out.println("Overridden toString():"+i1.toString());
        System.out.println("Overridden hashCode():"+i1.hashCode());
        char ch = 'a';
        Character c = new Character(ch);
        System.out.println("Non-primitive char:"+c);
        System.out.println("Overridden toString():"+c.toString());
        System.out.println("Overridden hashCode():"+c.hashCode());
        Boolean b = new Boolean(true);
        System.out.println("Overridden toString():"+b.toString());
    }
}
```

- *We can't compare object reference of two different objects.*
- *Because, both objects are different.*

Unboxing:

- To perform unboxing every wrapper class has a non-static method called **value**.

Example:

1. Integer Class → `intValue()`;
 2. Character Class → `charValue()`;
 3. Boolean Class → `booleanValue()`;
- These returns the primitive form of the boxed value.

Example:

```
package Wrapper;
public class UnBoxing {
    public static void main (String [] args) {
        Integer i = new Integer(10);
        int i1 = i.intValue();
        System.out.println("Primitive int:"+i1);
        Character c = new Character('e');
        char c1 = c.charValue();
        System.out.println("Primitive char:"+c1);
    }
}
```

Auto boxing & Auto Unboxing:

- From JDK 1.5 Boxing & Unboxing became implicit which was known as auto-boxing & auto-unboxing.

Example:

```
package Wrapper;

public class AutoBoxUnbox {
    public static void main (String [] args) {
        Integer i = 10; // Auto-Boxing
        int a = i; // Auto-Unboxing
        System.out.println(i.toString());
        System.out.println("Primitive int:"+a);
        Double d = 123.12; // Auto-Boxing
        double d1 = d; // Auto-Unboxing
        System.out.println(d.toString());
        System.out.println("Primitive double:"+d);
    }
}
```

Modifiers:

- *Modifiers are the keywords which are used to alter the behaviour of the Java Members.*
- *They are classified into 2 types –*
 - i.** *Access Modifiers*
 - ii.** *Non-Access Modifiers (Modifiers)*

1. Access Modifiers:

- *Access Modifiers are the keywords which alters the behaviour of Java Members according to visibility*
- *There are 4 Access Modifiers in Java –*
 - i.** *Public*
 - ii.** *Private*
 - iii.** *Protected*
 - iv.** *Default*

I. “Private” Access Modifier:

- *Private is a keyword & a modifier.*
- *Any member prefixed with private access modifier is known as Private Member.*
- *A member which is prefixed with private modifier cannot be accessed outside of the class.*
- *We cannot access private members in different class or subclass & parent class of same package.*
- *A private members cannot be accessed outside the package.*

NOTE:

- *Only global variable, methods & constructors are able to use Private Access Modifier.*

Example:

```
package Modifiers;
public class A {
    static private int a =10;
    private void test() {
        System.out.println("test() from A [Private]");
    }
    private A() {
    }
    public static void main (String [] args) {
        A a1 = new A();
        System.out.println(a1);
        a1.test();
    }
}
```

In Another Class:

```

package Modifiers;
public class B {
    public static void main (String [] args) {
        A a1 = new A(); //CTE
        System.out.println(A.a1);
        a1.test();
    }
}

```

II. “Default” Access Modifier:

- If a member is not prefixed with any modifier, it is known as a default member.
- We cannot use a default member outside of package.
- A class which is not prefixed with public (default class) cannot be accessed in any other package.

Example:

```

package Modifiers;
public class Default_A {
    static int a = 10;
    void test() {
        System.out.println("test() from A [Default]");
    }
    Default_A(){
    }
    public static void main (String [] args) {
        Default_A a = new Default_A();
        System.out.println(a);
        a.test();
    }
}

```

In Another Class:

```

package Modifiers;
public class CheckDefault_B {
    public static void main (String [] args) {
        Default_A a = new Default_A();
        System.out.println(a);
        a.test();
    }
}

```

Another Package:

```

package Modifiers2;
import Modifiers.Default_A;
public class AnotherPack {
    public static void main (String [] args) {
        Default_A a = new Default_A(); //CTE
        System.out.println(a);
        a.test();
    }
}

```

NOTE:

- If we made *Modifiers.Default_A* class default then we cannot import *Modifiers.Default_A* package inside another package.

Visibility:

- Default member can be used anywhere within same package but not outside of package.
- Visibility of class is –
 - Final*
 - Abstract*
 - Public*
 - Final*
- Class cannot be inherited that's why no **protected** and **private** is usable.

III. “Protected” Access Modifier:

- It is a keyword & a Modifier.
- Any member prefixed with *protected* keyword is known as a protected member.
- We can use protected member anywhere within the same package.
- We cannot use protected member outside the package only with the help of subclass.

Example:

```

package Modifiers;
public class Protected_A {
    protected void test() {
        System.out.println("test() from Protected_A [Protected]");
    }
    protected Protected_A() {

    }
    public static void main (String [] args) {
        protected_A a = new Protected_A();
        System.out.println(a);
    }
}

```

```

        a.test(); // Directly Usable
    }
}

```

Another Class:

```

package Modifiers;
public class CheckProtected_B {
    public static void main (String [] args) {
        Protected_A a = new Protected_A();
        System.out.println(a);
        a.test(); // Directly Usable
    }
}

```

In Another Package:

```

package Modifiers2;
import Modifiers.Protected_A;
public class Protected_Y extends Protected_A{
    public static void main (String [] args) {
        Protected_Y y = new Protected_Y();
        System.out.println(y);
        y.test();
    }
}

```

IV. “Public” Access Modifier:

- It is an access modifier or keyword.
- If a member is prefixed with public modifier is known as public member.
- We can use public member anywhere.

NOTE:

- We cannot make a class private or protected (Class can only be public, final, abstract & default).

	Class	Method	Variable	Constructor	Initializer
Public	True	True	True	True	False
Private	False	True	True	True	False
Protected	False	True	True	True	False
Default	True	True	True	True	False

	Private	Public	Protected	Default
Member inside of same class of same package	True	True	True	True
Different class of same package	False	True	True	True
Different class of different pack directly	False	True	False	False
Subclass of same package	False	True	True	True
Subclass of different package	False	True	True	False

NOTE:

- While overriding the visibility must be higher or equal as per ascending order.
- Access Modifier of overriding method must be greater or equal to Access Modifier of overridden method.

Final Modifier:

- Final is a keyword & modifier.
- We can prefix final modifier to a class, method and variable.

Final Variable:

- A variable prefix with final modifier is known as final modifier.
- Final variable behaves like constant (cannot be change / re-initialized / re-assigned).

NOTE:

- A local variable can be declared without initialization but it can be assigned with a value only once. Anywhere within the block before its use.

Example:

```
package FinalModifier;
public class FinalVar1 {
    public static void main (String [] args) {
        final int a = 10;
        System.out.println(a);

        //a = 20; //CTE
        System.out.println(a);
    }
}
```

Example:

```
package FinalModifier;
```

```

public class Final2 {
    public static void main (String [] args) {
        final int a ;
        a = 100;
        System.out.println(a); //100
        a = 10; //CTE
    }
}

```

Static (Global) Final variable:

- A static variable cannot be declared without initialization.
- But we can initialize directly using declared initialization statement or we can also initialize static block.

Example:

```

package FinalModifier;
public class Final3 {
    static final char ch = 'a';
    static final double d;
    static final String s = "Vinay";
    static {
        d = 22.0/7.0;
    }
    public static void main (String [] args) {
        System.out.println(ch);
        System.out.println(s);
        s = "Rushi"; //CTE
        System.out.println(d);
    }
}

```

Non-Static (Global) variable:

- We cannot declare non-static final variable with initialization.
- But we can declare and initialize directly using non-static initializer and constructor class.

Example:

Final Method:

- A method prefix with final modifier is known as Final method.
- A Final method can be inherited but cannot overridden.

Example:

Final Class:

- A class prefixed with Final modifier is known as Final Class.
- We make class Final because it cannot be inherited by any other class.
- **Example:** Wrapper Class, String Class, Scanner Class are Final Classes.

Arrays:

- Array is a data structure which is used to store multiple data.
- In Java, Array is a non-primitive data
- It is a Homogeneous collection of similar type of elements(data).
- Arrays are predefined in size.
- Arrays is used to store contiguous block of memory which is used to store multiple data.

NOTE:

- Arrays can store similar type of values.
- Arrays store and read with the help of index and array reference.
- Index of arrays start from 'zero' to 'array size - 1' (n-1).
- In Java, array is an object which is represented using an array operator [].
- The variable which is used to reference of array of object is known as "Array Reference Variable".

Syntax of Creating an Array:

new datatype[size];

Syntax of Creating Array Reference Variable:

Datatype [] variable;

Syntax for declared and initialize array reference variable:

Datatype [] variable = new datatype [size];

- New operator creates an object in Heap Area of the given type and specify its size, store it with default values and return reference of array object.
- The datatype specifier specifies what type of array is created.
- The size specifies the capacity of array
- To create an array of a class name then class must be created by this name.

NOTE:

- Index always starts from zero and remains positive from 0 to size-1.

ArrayIndexOutOfBoundsException:

- These occurs when we try to access array element with the help of array index which is greater than the length or less than zero.

Syntax to store value in Array:

arrayReference [index] = value / expression;

Example:

package Arrays;

```
public class Arr2 {
    public static void main (String [] args) {
        char [] ch = new char [4];
        ch[0] = 'a';
        ch[1] = 'b';
        ch[2] = 'c';
    }
}
```

```

        ch[3] = 'd';
        System.out.println(ch[0]);
        System.out.println(ch[1]);
        System.out.println(ch[2]);
        System.out.println(ch[3]);
    }
}

```

Finding length of given Array:

```

package Arrays;

public class Arr3 {
    public static void main (String [] args) {
        String [] st = new String[7];
        System.out.println(st.length);
    }
}

```

Iterating the Array:

```

package Arrays;

public class Arr4 {
    public static void main (String [] args) {
        int ar [] = new int [6];
        for (int i = 0; i < ar.length; i++) {
            System.out.println(ar[i]);
        }
    }
}

```

Ask User to Enter size of Array, Display element. After all data is entered, display names one by one:

```

package Arrays;

import java.util.Scanner;

public class Arr5 {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println("Enter the Number of Friends :");
        int n = sc.nextInt();
        System.out.println("Enter the Name of Friend :");
        String [] ar = new String[n];
        for (int i = 0; i < n; i++) {
            ar[i] = sc.next();
        }
    }
}

```

```

        for (int i = 0; i < n; i++) {
            System.out.println(ar[i]);
        }
    }
}

```

Another way of Creating the Array:

```

package Arrays;
public class Arr6 {
    public static void main (String [] args) {
        int [] ar = {10,20,30,40,50,60};
        System.out.println("Size of Array : "+ar.length);

        for (int i = 0; i < ar.length; i++) {
            System.out.println(ar[i]);
        }
    }
}

```

Even Array Program:

```

package Arrays;

public class Arr7 {
    public static void main (String [] args) {
        int ar [] = {10,20,1,9,2,12,3};
        for (int i = 0; i < ar.length; i++) {
            if (ar[i] % 2 == 0) {
                System.out.println(ar[i]);
            }
        }
    }
}

```

Odd Array Program:

```

package Arrays;
public class Arr7 {
    public static void main (String [] args) {
        int ar [] = {10,20,1,9,2,12,3};
        for (int i = 0; i < ar.length; i++) {
            if (ar[i] % 2 != 0) {
                System.out.println(ar[i]);
            }
        }
    }
}

```

Double Pointer Program (Making Pairs of Starting element and Ending element):

```
package Arrays;
public class Arr8 {
    public static void main (String [] args) {
        int ar[] = {10,1,9,2,12,11,3};
        for (int start = 0,end = ar.length - 1; start <= end; start++,end--) {
            if (start != end) {
                System.out.println(ar[start]+" "+ar[end]);
            }
            else {
                System.out.println(ar[start]);
            }
        }
    }
}
```

Reverse copy of an Array:

```
package Arrays;

import java.util.Arrays;
public class Arr9 {
    public static void main (String [] args) {
        int ar [] = {10,9,1,3,12,111};
        int temp[] = new int[ar.length];

        for (int i = 0; i < ar.length; i++) {
            temp[ar.length - 1 - i] = ar[i];
        }
        System.out.println(Arrays.toString(temp));
    }
}
```

Merge 2 Arrays in 2 Loops:

```
package Arrays;
import java.util.Arrays;

public class Arr10 {
    public static void main (String [] args) {
        int [] ar1 = {10,20,30,40,50,60};
        int [] ar2 = {70,80,90,100,110,120};

        int [] ar3 = new int[ar1.length + ar2.length];
    }
}
```

```

        for (int i = 0; i < ar1.length; i++) {
            ar3[i] = ar1[i];
        }
        for (int i = ar1.length, j = 0; i < ar3.length; i++, j++) {
            ar3[i] = ar2[j];
        }
        System.out.println(Arrays.toString(ar3));
    }
}

```

ZigZag:

```

package Arrays;
import java.util.Arrays;
public class Arr11 {
    public static void main (String [] args) {
        int [] ar1 = {10,20,30,40,50,60};
        int [] ar2 = {70,80,90,100,110,120};

        int [] ar3 = new int[ar1.length + ar2.length];
        for (int i = 0, j = 0; i < ar3.length; i++,j++) {
            ar3[i] = ar1[j];
            ar3[i + 1] = ar2[j];
            i++;
        }
        System.out.println(Arrays.toString(ar3));
    }
}

```

Reverse Array:

```

package Arrays;
import java.util.Arrays;
public class ReverseArray {
    public static void main (String [] args) {
        int ar [] = {10,20,30,40,50,60};
        for (int i = 0, j = ar.length - 1; i < j; i++, j--) {
            int temp = ar[i];
            ar[i] = ar[j];
            ar[j] = temp;
        }
        System.out.println(Arrays.toString(ar));
    }
}

```

Palindrome Array:

```
package Arrays;
import java.util.Arrays;

public class PalindromeArray {
    public static void main (String [] args) {
        int [] ar = {10,20,30,30,20,10};
        int i = 0, j = ar.length - 1;
        for (;i < j; i++, j--) {
            if (ar[i] != ar[j]) {
                break;
            }
        }
        if (i < j) {
            System.out.println("It is not a Palindrome");
        }
        else {
            System.out.println("It is Palindrome");
        }
    }
}
```

Merge Even Array:

```
package Arrays;
import java.util.Arrays;
public class MergeEvenArray {
    public static void main (String [] args) {
        int ar1[] = {10,20,30};
        int ar2[] = {40,50,60,70};

        int [] ar3 = new int[ar1.length + ar2.length];

        if (ar1.length < ar2.length) {
            int j = 0;
            for (int i = 0; i < ar1.length; i++, j+=2) {
                ar3[j] = ar1[i];
                ar3[j+1] = ar2[i];
            }
            for (int i = ar1.length; i < ar2.length; i++, j++) {
                ar3[j] = ar2[i];
            }
        }
        else if(ar1.length > ar2.length) {
```



```

        int j = 0;
        for (int i = 0; i < ar2.length; i++, j+=2) {
            ar3[j] = ar1[i];
            ar3[j+1] = ar2[i];
        }
        for (int i = ar2.length; i < ar1.length; i++, j++) {
            ar3[j] = ar1[i];
        }
    }
    System.out.println(Arrays.toString(ar3)); // [10,40,20,50,30,60,70]
}

```

Merge Uneven Array:

```

package Arrays;
import java.util.Arrays;
public class MergeUnevenArray {
    public static void main (String [] args) {
        int ar1[] = {1,2,3};
        int ar2[] = {10,20,30,40};

        int ar3[] = new int[ar1.length + ar2.length];

        if (ar1.length < ar2.length) {
            int i = 0, j = 0;
            while(j < ar3.length) {
                if (i < ar1.length) {
                    ar3[j] = ar1[i];
                    ar3[j+1] = ar2[i];
                    j+=2;
                    i++;
                }
                else {
                    ar3[j] = ar2[i];
                    i++;
                    j++;
                }
            }
        }
        else if(ar1.length > ar2.length) {
            int i = 0, j = 0;
            while(j < ar3.length) {
                if (i < ar2.length) {

```

```

                                ar3[j] = ar1[i];
                                ar3[j+1] = ar2[i];
                                j+=2;
                                i++;
                            }
                            else {
                                ar3[j] = ar2[i];
                                i++;
                                j++;
                            }
                        }
                    }
                }
                System.out.println(Arrays.toString(ar3));
            }
        }
    }

```

Right Rotate:

```

package Arrays;
import java.util.Arrays;
public class RightRotate {
    public static void main (String [] args) {
        int ar [] = {1,2,3,4,5};
        int temp = ar[ar.length - 1];

        for (int i = ar.length - 1; i > 0; i--) {
            ar[i] = ar[i-1];
            ar[i-1] = temp;
        }
        System.out.println(Arrays.toString(ar));
    }
}

```

Half Right Half Left:

```

package Arrays;
import java.util.Arrays;
public class HalfLHalfR {
    public static void main (String [] args) {
        int [] ar = {1,2,3,4,5};
        int first = ar[0];
        for (int i = 0; i < (ar.length/2) - 1; i++) {
            ar[i] = ar[i+1];
        }
        ar[(ar.length/2)-1] = first;
        int last = ar[ar.length - 1];
    }
}

```

```

        for (int i = ar.length - 1; i > ar.length/2; i--) {
            ar[i] = ar[i-1];
        }
        ar[(ar.length/2)+1] = last;
        System.out.println(Arrays.toString(ar));
    }
}

```

Insert New Element:

```

package Arrays;
import java.util.Arrays;
import java.util.Scanner;
public class InsertNewElement {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println("Enter the Number of Element to Insert:");
        int no = sc.nextInt();
        int ar [] = {1,2,3,4};
        int ans [] = new int[ar.length + no];
        System.out.println("Enter the Elements:");
        for (int i = 0; i < ans.length; i++) {
            if (i < ar.length) {
                ans [i] = ar[i];
            }
            else {
                ans[i] = sc.nextInt();
            }
        }
        System.out.println(Arrays.toString(ans));
    }
}

```

Output:

```

Enter the Number of Element to Insert:
2
Enter the Elements:
5
6
[1, 2, 3, 4, 5, 6]

```

Index to Insert in Array:

```

package Arrays;
import java.util.Arrays;
import java.util.Scanner;
public class IndexToInsert {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println("Enter the Index to Insert :");
        int index = sc.nextInt();
        int ar [] = {1,2,3,4,5,6,7,8,9};
        if (index >= 0 && index < ar.length) {
            int element = sc.nextInt();
            int ans [] = new int [ar.length +1];
            for (int i = 0; i < ans.length; i++) {
                if (i < index) {
                    ans[i] = ar[i];
                }
                else if (i == index) {
                    ans[i] = element;
                }
                else {
                    ans[i] = ar[i - 1];
                }
            }
            System.out.println(Arrays.toString(ans));
        }
    }
}

```

Output:

```

Enter the Index to Insert:
5
12
[1, 2, 3, 4, 5, 12, 6, 7, 8, 9]

```

Remove the Element in Between the Range:

```

package Arrays;
import java.util.Scanner;
import java.util.Arrays;
public class RemoveElement {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        int ar [] = {1,2,3,4,5,6};
        System.out.println("Enter the First Index");
    }
}

```

```

        int start = sc.nextInt();
        System.out.println("Enter the Last Index");
        int end = sc.nextInt();
        int ans [] = new int[ar.length - (end - start+1)];
        int ansIndex = 0;
        for (int i = 0; i < ar.length; i++) {
            if (i < start || i > end) {
                ans[ansIndex] = ar[i];
                ansIndex++;
            }
        }
        System.out.println(Arrays.toString(ans));
    }
}

```

Output:

Enter the First Index

1

Enter the Last Index

2

[1, 4, 5, 6]

Remove all Even Element from Array:

```

package Arrays;
import java.util.Arrays;
public class RemoveEvenElement {
    public static void main (String [] args) {
        int ar [] = {1,2,3,4,5,6,7,8};
        int count = 0;
        for (int i = 0; i < ar.length; i++) {
            if (ar[i] % 2 == 0) {
                count++;
            }
        }
        int ar2 [] = new int[ar.length - count];
        for (int i = 0, j = 0; i < ar.length; i++) {
            if (ar[i] % 2 != 0) {
                ar2[j] = ar[i];
                j++;
            }
        }
        System.out.println(Arrays.toString(ar2));
    }
}

```

Output:

[1, 3, 5, 7]

Make Prime OR Find Prime:

```
package Arrays;
import java.util.Arrays;
public class MakePrime {
    public static void main (String [] args) {
        int ar[] = {1,3,56,3,6,6,8,43,5,3,7};
        for (int i = 0; i < ar.length; i++) {
            ar[i] = nextPrime(ar[i]);
        }
        System.out.println(Arrays.toString(ar));
    }
    public static int nextPrime(int num) {
        int j;
        for (j = num; ; j++) {
            int i;
            for ( i = 2; i < j; i++) {
                if (j % i == 0) {
                    break;
                }
            }
            if (i == j) {
                break;
            }
        }
        return j;
    }
}
```

Output:

[2, 3, 59, 3, 7, 7, 11, 43, 5, 3, 7]

Count all the Elements:

```
package Arrays;
import java.util.Arrays;
public class CountElement {
    public static void main (String [] args) {
        int ar[] = {1,1,2,2,3,3};
        int count []= new int [ar.length];
        for (int i = 0; i < ar.length-1; i++) {
            int ct = 0;
```

```

        for (int j = 0; j < count.length; j++) {
            if (ar[i] == ar[j]) {
                ct++;
                count[j] = -1;
            }
        }
        if (count[i] != 1) {
            count[i] = ct;
        }
    }
    for (int i = 0; i < count.length; i++) {
        if (count[i] != -1) {
            System.out.println("Element "+ar[i]+" is repeated
"+count[i]+" times");
        }
    }
}

```

Output:*Element 1 is repeated 2 times**Element 2 is repeated 2 times**Element 3 is repeated 2 times***Optimized Count Elements:**

```

package Arrays;
import java.util.Arrays;
public class OptimizedCountElement {
    public static void main (String [] args) {
        int ar[] = {1,1,2,2,3,3};
        int count [] = new int[ar.length];
        for (int i = 0; i < ar.length; i++) {
            if (count[i] == -1) {
                continue; // Skip already counted elements
            }

            int ct = 1; // Initialize count for the current element

            for (int j = i + 1; j < ar.length; j++) {
                if (ar[i] == ar[j]) {
                    ct++;
                    count[j] = -1; // Mark duplicate elements
                }
            }
        }
    }
}

```

```

        count[i] = ct; // Store the count for the current element
    }
    for (int i = 0; i < count.length; i++) {
        if (count[i] != -1) {
            System.out.println("Element "+ar[i]+" is repeated
"+count[i]+" times");
        }
    }
}

```

Output:

Element 1 is repeated 2 times

Element 2 is repeated 2 times

Element 3 is repeated 2 times

- *Remove all the repeated Element (-1).*
- *Segregate array element into repeated and non-repeated array.*

Union & Intersection in Array:

An Array contains Positive & Negative, push all the positive to left & Negative to Right:

Find Length of Array without using length modifier:

```

package Arrays;

public class FindLength {
    public static void main (String [] args) {
        int ar[] = {1,2,3,4,5,6,7};
        int count = 0;
        try {
            for (int i = 0; ; i++) {
                ar[i] = ar[i];
                count++;
            }
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(count);
        }
    }
}

```

Output:

6

Maximum Element in Array:

```
package Arrays;

public class MaximumElement {
    public static void main (String [] args) {
        int [] ar = {1,2,3,4,5,6,7};
        int max = Integer.MIN_VALUE;
        for (int i = 0; i < ar.length; i++) {
            if (ar[i] > max) {
                max = ar[i];
            }
        }
        System.out.println(max);
    }
}
```

Output:

7

Minimum Element in Array:

```
package Arrays;

public class MinimumElement {
    public static void main (String [] args) {
        int ar[] = {1,2,3,4,5,6,7};
        int min = Integer.MAX_VALUE;
        for (int i = 0; i < ar.length; i++) {
            if (ar[i] < min) {
                min = ar[i];
            }
        }
        System.out.println(min);
    }
}
```

Output:

1

Second Maximum Element in Array:

```
package Arrays;

public class SecondMaxElement {
    public static void main (String [] args) {
        int [] ar = {1,2,3,4,5,6,7};
        int max = Integer.MIN_VALUE, secMax = Integer.MIN_VALUE;

        for (int i = 0; i < ar.length; i++) {
```

```

        if (ar[i] > max) {
            secMax = max;
            max = ar[i];
        }
        else if (ar[i] > secMax && ar[i] != max) {
            secMax = ar[i];
        }
    }
    System.out.println(secMax);
}
}

```

Output:

6

Second Minimum Element in Array:

```
package Arrays;
```

```

public class SecondMinElement {
    public static void main (String [] args) {
        int ar[] = {1,2,3,4,5,6,7};
        int min = Integer.MAX_VALUE, secMin = Integer.MAX_VALUE;

        for (int i = 0; i < ar.length; i++) {
            if (ar[i] < min) {

                secMin = min;
                min = ar[i];
            }
            else if (ar[i] < secMin && ar[i] != min) {
                secMin = ar[i];
            }
        }
        System.out.println(secMin);
    }
}

```

Output:

2

Check Element of First Array is present in Second Array:

```
package Arrays;
```

```

public class Check1stInto2nd {
    public static void main (String [] args) {

```

```

int [] a = {'a','b','c','d'};
int [] b = {'a','b','c','d'};

if (a.length == b.length) {
    int i;
    for ( i = 0; i < b.length; i++) {
        int j;
        for ( j = 0; j < b.length; j++) {
            if (a[i] == b[j]) {
                break;
            }
        }
        if (j == b.length) {
            break;
        }
    }
    if (i == a.length) {
        System.out.println("It is an Anagram Array!");
    }
    else {
        System.out.println("It is NOT an Anagram Array!");
    }
}
else {
    System.out.println("It is NOT an Anagram Array!");
}
}
}

```

Output:

It is an Anagram Array!

Two-Dimensional Array:

```

package Arrays;
import java.util.Scanner;
public class TwoDimensional {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        int [][] ar = new int[3][3];
        for (int i = 0; i < ar.length; i++) {
            for (int j = 0; j < ar[0].length; j++) {
                ar[i][j] = sc.nextInt();
            }
        }
    }
}

```

```

    }
    for (int i = 0; i < ar.length; i++) {
        for (int j = 0; j < ar[0].length; j++) {
            System.out.print(ar[i][j]);
        }
        System.out.println();
    }
}

```

Output:

```

1
2
3
4
5
6
7
8
9
123
456
789

```

Reverse Two-Dimensional Array:

```
package Arrays;
```

```

public class Reverse2D {
    public static void main (String [] args) {
        int [][] ar = {{1,2,3},{4,5,6},{7,8,9}};
        reverse(ar);
    }
    public static void display(int [][]ar) {
        for (int i = 0; i < ar.length; i++) {
            for (int j = 0; j < ar.length; j++) {
                System.out.print(ar[i][j]+" ");
            }
            System.out.println();
        }
    }
    public static void reverse(int [][]ar) {
        for (int i = 0; i < ar.length; i++) {
            int [] temp = ar[i];
            for (int j = 0, k = temp.length-1; j < k; j++, k--) {

```

```

        int x = temp[j];
        temp[j] = temp[k];
        temp[k] = x;
    }
    ar[i] = temp;
}
display(ar);
}
}

```

Output:

3,2,1,
6,5,4,
9,8,7,

Find Duplicate & Missing:

```

package Arrays;
import java.util.Scanner;
public class FindDuplicateAndMissing {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println("Enter the Number of Element:");
        int n = sc.nextInt();
        int [] nums = new int[n];
        System.out.println("Enter the Elements:");
        for (int i = 0; i < n; i++) {
            nums[i] = sc.nextInt();
        }
        int [] ans = findDuplicateAndMissing(nums);
        System.out.println("Duplicate :"+ans[0]);
        System.out.println("Missing :"+ans[1]);
    }
    public static int[] findDuplicateAndMissing(int [] nums) {
        int n = nums.length;
        long sumN = (long)n * (n+1)/2;
        long sumNsq = (long)n * (n+1)*(2*n+1)/6;
        long sum = 0, sumSq = 0;
        for (int num:nums) {
            sum+= num;
            sumSq = (long) num*num;
        }
        long diff = sumN - sum; //Missing - Duplicate
        long sumDiff = (sumNsq - sumSq)/diff; // Missing + Duplicate
        int missing = (int)(diff + sumDiff) / 2;
    }
}

```

```

        int duplicate = (int)(sumDiff - missing);
        return new int[] {duplicate,missing};
    }
}

```

Output:

Enter the Number of Element:

5

Enter the Elements:

11

11

2

3

4

Duplicate :7

Missing :9

Merge Two Array & put int Ascending Order:

```

package Arrays;
import java.util.Arrays;
public class MergeAndAscending {
    public static void main (String [] args) {
        int [] a = {1,2,30,40,50,100,234};
        int [] b = {12,56,78,90,102};

        int [] c = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length) {
            if (a[i] < b[j]) {
                c[k] = a[i];
                i++;
            } else {
                c[k] = b[j];
                j++;
            }
            k++;
        }

        // Copy remaining elements from a (if any)
        while (i < a.length) {
            c[k] = a[i];
            i++;
            k++;
        }
    }
}

```

```

    }

    // Copy remaining elements from b (if any)
    while (j < b.length) {
        c[k] = b[j];
        j++;
        k++;
    }

    System.out.println(Arrays.toString(c));
}
}

```

Output:

[1, 2, 12, 30, 40, 50, 56, 78, 90, 100, 102, 234]

Bubble Sort:

```

package Arrays;
import java.util.Arrays;
public class BubbleSort {
    public static void main (String [] args) {
        int [] ar = {1,3,53,67,33,2,7};
        for (int i = 0; i < ar.length-1; i++) {
            for (int j = 0; j < ar.length-1; j++) {
                if (ar[j] > ar[j+1]) {
                    int temp = ar[j];
                    ar[j] = ar[j+1];
                    ar[j+1] = temp;
                }
            }
        }
        System.out.println(Arrays.toString(ar));
    }
}

```

Output:

[1, 2, 3, 7, 33, 53, 67]

Binary Search:

```
package Arrays;
import java.util.Scanner;
import java.util.Arrays;
public class BinarySearch {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        int [] ar = {1,2,4,5,56,5676,2345};
        Arrays.sort(ar);
        System.out.print("Enter the Element to Search : ");
        int search = sc.nextInt();
        int start = 0, end = ar.length -1;
        int mid = (start + end) / 2;
        while (start < end) {
            if (ar[mid] == search) {
                System.out.println("Element found at index no
"+mid+".");
                return;
            }
            else if (search > ar[mid]) {
                start = mid + 1;
            }
            else {
                end = mid - 1;
            }
            mid = (start + end) / 2;
        }
        System.out.println("Element not found in the array.");
    }
}
```

Output:

```
Enter the Element to Search: 5
Element found at index no 3.
```


String Programs:**Iterate the String:**

```

package String
public class Str1 {
    public static void main (String [] args) {
        String st = "Vinay";
        for (int i = 0; i < st.length(); i++) {
            System.out.println(st.charAt(i));
        }
    }
}

```

Output:

V
i
n
a
y

Sort the String:

```

package String;
import java.util.Arrays;
public class SortString {
    public static void main (String [] args) {
        String st = "vinay";
        char ch[] = new char[st.length()];
        for (int i = 0; i < ch.length; i++) {
            ch[i] = st.charAt(i);
        }
        Arrays.sort(ch);
        System.out.println(ch);
    }
}

```

Output:

ainvy

Reverse the String:

```

package String;
public class ReverseString {
    public static void main (String [] args) {
        String st = "Vinay";
        String ans = "";
        for (int i = st.length()-1; i >= 0; i--) {
            ans += st.charAt(i);
        }
    }
}

```

```

    }
    System.out.println(ans);
}
}

```

Output:

yaniv

Search Character in String:

```

package String;
public class SearchChar {
    public static void main (String [] args) {
        String st = "Vinay";
        char search = 'a';
        int i;
        for ( i = 0; i < st.length(); i++) {
            if (st.charAt(i) == search) {
                break;
            }
        }
        if (i == st.length()) {
            System.out.println("Not Found");
        }
        else {
            System.out.println("Fount at " + (i+1) + " th Position");
        }
    }
}

```

Output:

Fount at 4 th Position

Remove Character from String:

```

package String;

public class RemoveChar {
    public static void main (String [] args) {
        String st = "Vinay";
        char search = 'i';
        String ans = "";
        int i;
        for (i = 0; i < st.length(); i++) {
            if (st.charAt(i) != search) {
                ans += st.charAt(i);
            }
        }
    }
}

```

```

        }
        System.out.println(ans);
    }
}

```

Output:

Vnay

Remove String using Replace Function:

```

package String;
public class RemoveString {
    public static void main (String [] args) {
        String st = "Vinay";
        String search = "nay";
        String ans = st.replace(search, " ");
        System.out.println(ans);
    }
}

```

Output:

Vi

Check words of String:

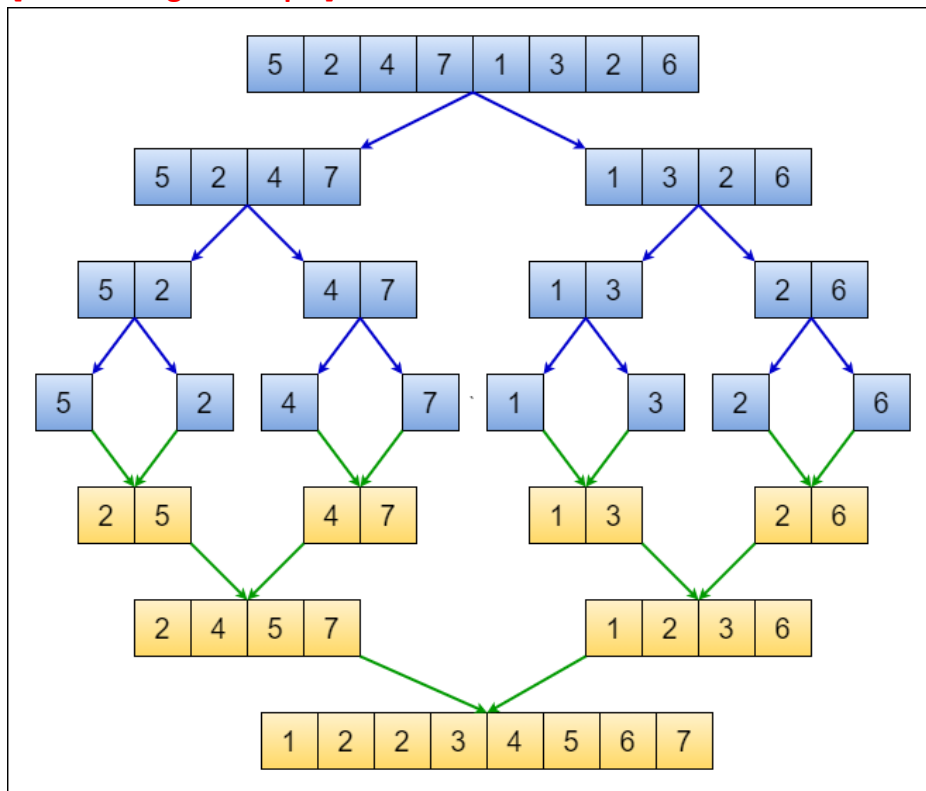
```

package String;
public class CheckWords {
    public static void main (String [] args) {
        String st = "Java is Very Boring";
        int count = 0;
        for (int i = 0; i < st.length(); i++) {
            if (st.charAt(i) == ' ') {
                count++;
            }
        }
        System.out.println((count++)+1+" words are there in the Sentence.");
    }
}

```

Output:

4 words are there in the Sentence.

Merge Sort: [Best Sorting Technique]**Program of Merge Sort:**

```

package String;
public class MergeSort {
    public static void merge(int []ar, int start, int mid, int end) {
        int ans[] = new int[ar.length];
        int i = start, j = mid+1, k = start;
        while (i <= mid && j <= end) {
            if (ar[i] < ar[j]) {
                ans[k] = ar[i];
                i++;
                k++;
            }
            else {
                ans[k] = ar[j];
                j++;
                k++;
            }
        }
        while (i <= mid) {
            ans[k] = ar[i];
            i++;
            k++;
        }
    }
}

```

```
        while (j <= end) {
            ans[k] = ar[j];
            j++;
            k++;
        }
        for (int k2 = start; k2 <= end; k2++) {
            ar[k2] = ans[k2];
        }
    }
    public static void mergeSort(int [] ar, int start, int end) {
        if(start < end) {
            int mid = (start+end)/2;
            mergeSort(ar,start,mid);
            mergeSort(ar,mid+1,end);
            merge(ar,start,mid,end);
        }
    }
    public static void main (String [] args) {
        int a [] = {7,8,5,4,3,1,2};
        mergeSort(a,0, a.length-1);
        System.out.println(Arrays.toString(a));
    }
}
```

Output:

[1, 2, 3, 4, 5, 7, 8]

Strings:

- Groups of character is known as String, and they are enclosed in double quotes("").
- String is built-in class in Java, present in java.lang Package.
- If we try to share a String, the String value stores in the object.
- We can store String objects in different ways.
 - i. Java.lang.String Class.
 - ii. Java.lang.StringBuilder Class.
 - iii. Java.lang.StringBuffer Class.

String Class:

- It is present in java.lang package.
- In Java, we can use String class for storing Strings.

NOTE:

1. String class is Public.
2. String class is Final.
3. String class extends to java.lang.Object Class
4. String class implements Serializable, Comparable & CharSequence Interface.
5. toString(), equals() and hashCode() methods of Object class are overridden by String Class.

Constructor of String Class:**A. Default Constructor:**

- It is used to create empty String Object.

B. String (String Original):

- Creates a new String Object & initialize it with a given String.

C. String (StringBuffer buffer):

- Creates a new String object & initialize with given StringBuffer value (used to convert StringBuffer to String)

D. String (StringBuilder builder):

- Creates a new String object & initialize with given StringBuilder value (used to convert StringBuilder to String)

E. String (char [] value):

- Creates a new String object & initialize with given character array (used to create char [] to String)

F. String (byte [] bytes):

- Used to convert a byte Array into String.

Program:

```
package String;  
public class Str2 {  
    public static void main (String [] args) {  
        String st1 = new String("Hello");  
        System.out.println(st1);  
        String st2 = new String("Hello");
```

```

        System.out.println(st2);
        System.out.println(st1 == st2);
        System.out.println(st1.equals(st2));
        System.out.println(st1.hashCode() + " , " + st2.hashCode());
    }
}

```

Output:

```

Hello
Hello
false
true
69609650
69609650

```

- We can create String Object in two ways.
 - i. By initializing the String literal directly.
 - ii. By using the new operator and constructor of class.
- If we create a String Object using String literal, the object gets created in **String Pool Area** which is present within heap area, it will not create directly in heap area.
- If the string given literal is already present in pool area, then no new object is created, instead of creating new object, it returns the address of the already present String Object.

Program:

```

package Strings;
public class Str3 {
    public static void main (String [] args) {
        String st1 = "Hello";
        String st2 = "Hello";
        System.out.println(st1 == st2);
        System.out.println(st1.equals(st2));
        System.out.println(st1.hashCode());
        System.out.println(st2.hashCode());
    }
}

```

Output:

```

true
true
69609650
69609650

```

Creating String Object using new Operator & Constructor:

- When we create object using new operator & constructor, the object gets created in heap area but not in String Pool Area.
- Every String object here is uniquely created in heap area.

Immutability of String (Characteristics of String):

- Objects of String Class are immutable (not changeable).
- Once the object gets created it can't be modified.
- When we try to modify the Object is created and reference is restored.

Program:

```
package Strings;
import java.util.Arrays;
public class Str4 {
    public static void main (String [] args) {
        String s = "India is my country & it has beautiful country-side view";
        System.out.println(s.charAt(14)); // u
        System.out.println(s.concat("so, I like it")); // India is my country & it
        has beautiful country-side view so, I like it

        System.out.println(s.contains("pak")); //false
        System.out.println(s.endsWith("beautiful")); //false
        String s1 = "heLlo";
        System.out.println(s1.equals("hello")); //false
        System.out.println(s1.equalsIgnoreCase("hello")); //true
        System.out.println(s.indexOf('z')); //-1
        System.out.println(s.indexOf("my state")); //-1
        System.out.println(s.indexOf("country", 17)); //39
        System.out.println(s.isEmpty()); //false
        System.out.println(s.lastIndexOf("country")); // 39
        System.out.println(s.length()); //56
        System.out.println(s.replace('a', '@')); // Indi@ is my country & it h@s
        be@utiful country-side view

        System.out.println(s.replace("country", "COUNTRY")); // India is my
        COUNTRY & it has beautiful COUNTRY-side view
        System.out.println(s.replaceFirst("country", "COUNTRY")); // India is
        my COUNTRY & it has beautiful country-side view

        String [] ar = s.split("a");
        System.out.println(s.substring(10)); // y country & it has beautiful
        country-side view

        System.out.println(s.substring(10,30)); // y country & it has b
```



```
char ch [] =s.toCharArray();  
System.out.println(Arrays.toString(ch)); // [l, n, d, i, a, , i, s, , m, y, ,  
c, o, u, n, t, r, y, , &, , i, t, , h, a, s, , b, e, a, u, t, i, f, u, l, , c, o, u, n, t, r, y, -, s, i, d, e,  
, v, i, e, w]
```

```
System.out.println(s.toLowerCase()); // india is my country & it has  
beautiful country-side view
```

```
System.err.println(s.toUpperCase()); // INDIA IS MY COUNTRY & IT  
HAS BEAUTIFUL COUNTRY-SIDE VIEW
```

```
System.out.println(" Vinay ".trim()); //Vinay  
}  
}
```

String Program:**Extract the Character:**

```
package Strings;

public class ExtractChar {
    public static void main (String [] args) {
        String s = "Vinay";
        for (int i = 0; i < s.length(); i++) {
            System.out.println(s.charAt(i));
        }
    }
}
```

Output:

V
i
n
a
y

Convert Array into String:

```
package Strings;
import java.util.Arrays;

public class ConvertArrToStr {
    public static char[] stringToChar(String s) {
        char [] ch = new char[s.length()];
        for (int i = 0; i < ch.length; i++) {
            ch[i] = s.charAt(i);
        }
        return ch;
    }
    public static void main (String [] args) {
        String ip = "Vinay";
        char [] ans = stringToChar(ip);
        System.out.println(Arrays.toString(ans));
    }
}
```

Output:

[V, i, n, a, y]

Sort the char of Given String:

```

package Strings;

public class SortChar {
    public static void main (String [] args) {
        String s = "vinay";
        System.out.println(sort(s));
    }
    public static String sort(String ip) {
        char [] ch = StringToArray.stringToCharArray(ip); // from another class
        for (int i = 0; i < ch.length-1; i++) {
            for (int j = 0; j < ch.length-1; j++) {
                if (ch[j] > ch[j+1]) {
                    char temp = ch[j];
                    ch[j] = ch[j+1];
                    ch[j+1] = temp;
                }
            }
        }
        return new String(ch);
    }
}

```

Output:**ainvy****Palindrome String:**

```

package Strings;

public class PalindromeString {
    public static void main (String [] args) {
        String s = "malayalam";
        int i = 0, j = s.length()-1;
        for (i = 0; i < j; i++,j--) {
            if (s.charAt(i) != s.charAt(j)) {
                break;
            }
        }
        if (i >= j) {
            System.out.println("Palindrome");
        }
        else {
            System.out.println("Not Palindrome");
        }
    }
}

```

```
    }
}
```

Output:

Palindrome

Reverse the String:

```
package Strings;

public class ReverseStringLogic {
    public static void main (String [] args) {
        String s = "abc";
        String ans = "";
        for (int i = s.length()-1; i >= 0; i--) {
            ans += s.charAt(i);
        }
        System.out.println(ans);
    }
}
```

Output:

cba

Convert the String into Uppercase:

```
package Strings;

public class UpperString {
    public static void main (String [] args) {
        String s = "vInay";
        String ans = "";
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch >= 'a' && ch <= 'z') {
                ans += (char)(ch-32);
            } else {
                ans += ch;
            }
        }
        System.out.println(ans);
    }
}
```

Output:

VINAY

Convert the String into Lowercase:

```
package Strings;

public class LowerString {
```

```

    public static void main (String [] args) {
        String s = "vInay";
        String ans = "";
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch >= 'A' && ch <= 'Z') {
                ans += (char)(ch+32);
            } else {
                ans += ch;
            }
        }
        System.out.println(ans);
    }
}

```

Output:

vinay

Replace the Character from String:

```

package Strings;
public class ReplaceCharacter {
    public static void main (String [] args) {
        String s = "Vinay";
        System.out.println(replace(s,'i','e'));
    }
    public static String replace(String ip, char old, char replace) {
        String ans = "";
        for (int i = 0; i < ip.length(); i++) {
            char ch = ip.charAt(i);
            if (ch == old) {
                ans += replace;
            }
            else {
                ans += ch;
            }
        }
        return ans;
    }
}

```

Output:

Venay

Convert a sentence into Array: [use Split() method by String not with char]

```
package Strings;
import java.util.Arrays;
public class SentenceToArray {
    public static void main (String [] args) {
        String s = "India is my Country.";
        String ans [] = stringToArray(s);
        System.out.println(Arrays.toString(ans));
    }
    public static String [] stringToArray(String ip) {
        String [] words = new String[countWords(ip)];

        for (int i = 0, j = 0; i < words.length; i++) {
            String temp = "";
            for (; j < ip.length(); j++) {
                char ch = ip.charAt(j);
                if (ch == ' ') {
                    j++;
                    break;
                }
                else {
                    temp += ch ;
                }
            }
            words[i] = temp;
            temp = "";
        }
        return words;
    }
    public static int countWords(String ip) {
        int count = 1;
        for (int i = 0; i < ip.length(); i++) {
            if (ip.charAt(i) == ' ') {
                count++;
            }
        }
        return count;
    }
}
```

Output:

[India, is, my, Country.]

Check if Two Strings are Anagram or Not:

```
package Strings;
import java.util.Arrays;
public class AnagramString {
    public static void main (String [] args) {
        String s1 = "god";
        String s2 = "dog"
        if (s1.length() == s2.length()) {
            char [] ch1 = s1.toCharArray();
            char [] ch2 = s2.toCharArray();
            Arrays.sort(ch1);
            Arrays.sort(ch2);
            int i = 0;
            for ( ; i < ch2.length; i++) {
                if (ch1[i] != ch2[i]) {
                    break;
                }
            }
            if (i == s2.length()) {
                System.out.println("It is an Anagram");
            }
            else {
                System.out.println("It is not an Anagram");
            }
        }
        else {
            System.out.println("It is not an Anagram");
        }
    }
}
```

Output:

It is an Anagram

Validate an IP Address:

```

package Strings;
import java.util.Arrays;
import java.lang.String;
public class IpValidation {
    public static void main (String [] args) {
        String ip = "12.168.1.10";
        String [] nums = ip.split("\\.");
        /*
         * The delimiter you're using is "\\.".
         * Let's break that down further:
         * The double backslash (\\) is an escape sequence in Java.
         * It represents a single backslash character.
         * The dot (.) is a special character in regular expressions, so you need
         * to escape it with the backslash.
         * Therefore, "\\." represents the literal dot character.
         */
        if (nums.length == 4) {
            try {
                int i = 0;
                for (; i < nums.length; i++) {
                    int num = Integer.parseInt(nums[i]);
                    if (nums[i].length() > 4) {
                        System.out.println("Not a Valid IP");
                        break;
                    }
                }
                if (i == nums.length) {
                    System.out.println("It is Valid IP");
                }
            }
            catch (NumberFormatException nfe) {
                System.out.println("Not a Valid IP");
            }
        }
        else {
            System.out.println("Not a Valid IP");
        }
    }
}

```

Output:**It is Valid IP**

Sum of Numbers in String:

```
package Strings;

public class SumOfNum {
    public static void main (String [] args) {
        String s = "abcd12345e6f7ghij";
        int sum = 0;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch >= '0' && ch <= '9') {
                sum += (ch-48);
            }
        }
        System.out.println(sum);
    }
}
```

Output:

28

StringBuilder & StringBuffer Classes:

- *StringBuilder & StringBuffer classes are present in java.lang package.*
- *These classes are used to store Strings.*
- *We use these classes to create Strings which are mutable (changeable / alterable).*
- *String Builder & Buffer classes do not override equals & hashCode methods of object class.*
- *It only overrides toString() method of Object Class.*

Constructor of String Builder:**StringBuilder ():**

- *Constructs a String Builder with no character in it and capacity of **16** Characters.*

StringBuilder (String str):

- *Constructs a String Builder initialized to the contents at the specified String (Constructor is used to convert String to StringBuilder).*

Constructor of StringBuffer:**StringBuffer ():**

- *Constructs a String Buffer with no character in it and capacity of **16** character.*

StringBuffer (String str):

- *Constructs a String Buffer initialized to the contents at the specified String (Constructor is used to convert String to StringBuffer).*

Create a StringBuilder:

```
package StringBuilder;
public class SB1 {
    public static void main (String [] args) {
        StringBuilder s0 = "Hello"; //CTE
        StringBuilder s1 = new
        StringBuilder("Hello");
        StringBuilder s2 = new
        StringBuilder("Hello");

        System.out.println(s1); //Hello
        System.out.println(s2); //Hello
        System.out.println(s1 == s2); //false
    }
}
```

Join a StringBuilder:

```
package StringBuilder;
public class SB2 {
    public static void main (String [] args) {
        StringBuilder sb1 = new
        StringBuilder("Qspiders");
        StringBuilder sb2 =
        sb1.append("Deccan");

        System.out.println(sb1); // QspidersDeccan
        System.out.println(sb2); // QspidersDeccan
        System.out.println(sb1 == sb2); // true

        String s1 = "Jspiders";
        String s2 = s1.concat("Deccan");

        System.out.println(s1); //Jspiders
        System.out.println(s2); //JspidersDeccan

        System.out.println(s1 == s2); //false
    }
}
```

Collection Framework:

- Collection Framework is a set of classes and interfaces which are group together, which provides a mechanism to store group of elements (Object Together).

Que. Why we need Collection Framework?

- To store multiple objects together we can generally use Arrays. But Arrays have some limitations. That is why we need Collection Framework.

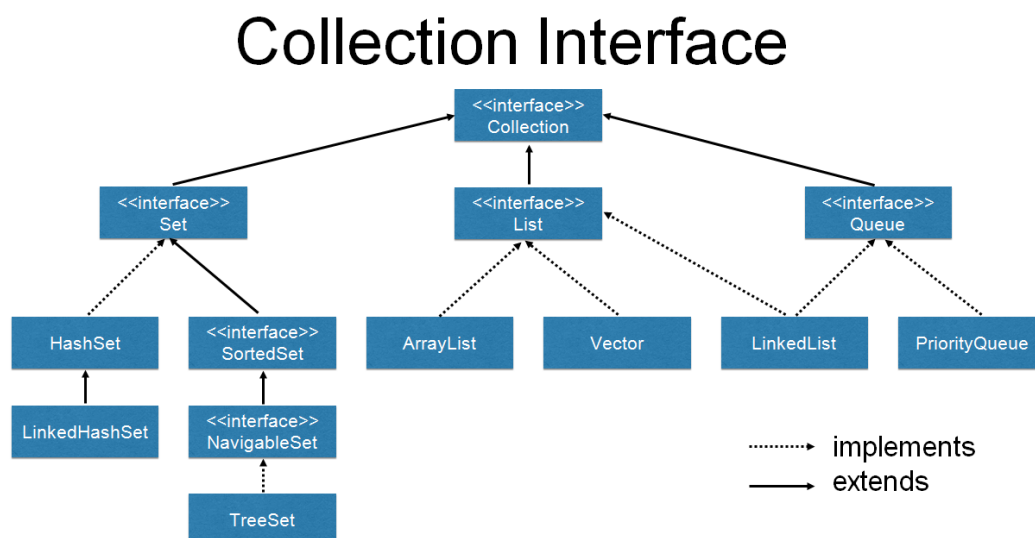
Limitations of Arrays:

1. Size of Array is fixed, we cannot increase or decrease size of an array dynamically during execution.
2. Array is collection of Homogeneous elements.
3. Array manipulation such as –
 - a. Removing the element, inserting an element require complex logics to solve.
 - b. To avoid limitation of array, we can store group object or element using different structures.
 - c. Such as –
 - i. **List**
 - ii. **Set**
 - iii. **Queue**
 - iv. **Map / Dictionaries.**

Collection Framework provide us mechanism to perform an action such as –
Create & Add element, Access the element, Remove or Delete element,
Search the element, Update the element Sort the element.

NOTE:

- There are 2 important hierarchies –
 - i. **Collection Hierarchy**
 - ii. **Map Hierarchy**

Collection Hierarchy:

Collection Interface:

- *Collection is an interface present in java.util Package.*
- *Collection Interface is a Base Interface or Root Interface of Collection Hierarchy.*
- *Collection Interface provides mechanism to store group of elements (Object) together.*
- *All the elements in a collection are stored in the form of Objects (Primitive data types are not allowed)*
- *Collection Interface provides abstract methods to the programmer to perform the following tasks –*
 - i. *Adding element into the collection*
 - ii. *Searching element in the collection*
 - iii. *Removing the element from collection*
 - iv. *Accessing or reading the elements present in the collection.*

Methods in the Collection Framework

Operations	Return Type	Method Signature
Adding Element	<i>boolean</i> <i>boolean</i>	<i>add(Object o)</i> <i>addAll(Collection c)</i>
Removing the Element	<i>boolean</i> <i>boolean</i> <i>boolean</i> <i>void</i>	<i>remove(Object o)</i> <i>removeAll(Collection c)</i> <i>retainAll(Collection c)</i> <i>clear()</i>
Accessing the Element	<i>Iterator</i> <i>//Loop</i>	<i>Iterator()</i> <i>forEachLoop()</i>
Search the Element	<i>boolean</i> <i>boolean</i>	<i>contains(Object o)</i> <i>containsAll(Collection c)</i>
Miscellaneous Operations	<i>boolean</i> <i>Object []</i> <i>boolean</i> <i>int</i> <i>int</i>	<i>isEmpty()</i> <i>toArray()</i> <i>equals(Object o)</i> <i>hashCode()</i> <i>size()</i>

List Interface:

- *List is a sub-interface present in **java.util** Package.*
- *It inherits all the methods of collection Interface.*

Characteristics:

1. *List is a Collection of Objects.*
2. *List maintains the insertion order of elements. That is, the element inserted first, will be maintained in the first position.*
3. *It allows duplicate entries. That is, we can add same elements (Object) multiple times.*
4. *List supports Indexing. We can insert, search, remove, access elements with the help of Index.*
5. *The index starts from zero till (list size – 1).*
6. *List allows multiple null values.*

ArrayList:

- *ArrayList is a Concrete implementing class of **List Interface**.*

Characteristics of ArrayList is same as List.

- *ArrayList is defined in java.util package.*

NOTE:

- *All abstract methods of List & Collection Interface are implemented by ArrayList.*
- *We can create Object of ArrayList.*

Constructors of ArrayList:**1. ArrayList():**

- *Creates an Empty ArrayList of initial capacity **10**.*

2. ArrayList (Collection c):

- *Creates an ArrayList object and copies all the elements present in the given collection into the ArrayList.*

NOTE:

- *toString() method is overridden in such way that it returns the view of all the elements present in ArrayList*

Ex: [ele1, ele2, ele3, ...]

Implementation:

```
ArrayList a1 = new ArrayList<>();
a1.add(10);
a1.add("Hello");
a1.add('a');
a1.add("RIP");
```

Program:**Store and Add elements:**

```
package Collection;
import java.util.ArrayList;
public class ArrayList1 {
    public static void main (String [] args) {
```

```

        ArrayList a1 = new ArrayList();
        a1.add(10);
        a1.add("Hello");
        a1.add(10.20);
        a1.add(true);
        a1.add(123);
        a1.add(123);
        for (int i = 0; i < a1.size(); i++) {
            System.out.println(a1.get(i));
        }
    }
}

```

Output:

```

10
Hello
10.2
true
123
123

```

For Each Loop:

- It is used to iterate over a collection or an Array.

Syntax:

```

for (var type of element in collection / array: reference of array / collection) {
    // statements;
    // statements;
}

```

Non-Primitive Array Creation:

Student Blueprint:

```

package Collection.arrayList;
public class Student {
    String name;
    int year;
    String stream;
    public Student(String name, int year, String stream) {
        this.name = name;
        this.year = year;
        this.stream = stream;
    }
    public String toString() {
        return "[Name :"+name+", Year :"+year+", Stream :"+stream+"]";
    }
}

```

Student Driver:

```

package Collection.arrayList;
import java.util.Arrays;
public class StudentDriver {
    public static void main (String [] args) {
        Student s [] = new Student [4];
        s[0] = new Student("Raju",3,"CS");
        s[1] = new Student("Rani",3,"CS");
        s[2] = new Student("Gulam",4,"CS");
        System.out.println(Arrays.toString(s));
        System.out.println(s);
    }
}

```

Output:

```

[[Name: Raju, Year :3, Stream: CS], [Name: Rani, Year :3, Stream: CS],
[Name: Gulam, Year :4, Stream: CS], null]
[Ljava.util.ArrayList$Student;@e9e54c2

```

Driver Class using ArrayList:

```

package Collection.arrayList;
import java.util.ArrayList;
public class StudentArrayListDriver {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add(new Student("Raju",3,"CS"));
        a1.add(new Student("Rani",3,"CS"));

        for (Object o : a1) {
            Student s = (Student)o; // DownCasting reference from Object
            to Student
            System.out.println(s.name); // Prints each element in array
            itself
        }
    }
}

```

Output:

```

Raju
Rani

```

Adding Two Collections:

```
package Collection.arrayList;
import java.util.ArrayList;
public class AddCollections {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add("Apple");
        a1.add("Mango");
        a1.add("Banana");
        a1.add("Grapes");
        a1.add("Chickoo");

        ArrayList a2 = new ArrayList();
        a2.add("Tomato");
        a2.add("Potato");
        a2.add("Spinach");
        a2.add("Lady's Finger");
        a2.add("Carrot");
        ArrayList a3 = new ArrayList();
        a3.addAll(a1);
        a3.addAll(a2);
        System.out.println(a3);
    }
}
```

Output:

[Apple, Mango, Banana, Grapes, Chickoo, Tomato, Potato, Spinach, Lady's Finger, Carrot]

Add all elements from a2 to a1 from index 2:

```
package Collection.arrayList;
import java.util.ArrayList;
public class AddCollections2 {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add("Apple");
        a1.add("Mango");
        a1.add("Banana");
        a1.add("Grapes");
        a1.add("Chickoo");
        ArrayList a2 = new ArrayList();
        a2.add("Tomato");
        a2.add("Potato");
```



```

        a2.add("Spinach");
        a2.add("Lady's Finger");
        a2.add("Carrot");
        a1.addAll(2, a2); // Adds elements of A2 to A1 from index 2
        System.out.println(a1);
    }
}

```

Output:

[Apple, Mango, Tomato, Potato, Spinach, Lady's Finger, Carrot, Banana, Grapes, Chickoo]

Remove Array from ArrayList:

```

package Collection.arrayList;
import java.util.ArrayList;
public class RemoveObject {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add(10);
        a1.add(20);
        a1.add(30);
        a1.add(40);
        a1.add(50);
        a1.remove((Integer)20); // Up-casts element to Object & pass to
remove()

        System.out.println(a1);
    }
}

```

Output:

[10, 30, 40, 50]

Display Elements which Starts and Ends with "O":

```

package Collection.arrayList;
import java.util.ArrayList;
public class RemoveElement {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add("Potato");
        a1.add("Orange");
        a1.add("Tomato");
        a1.add("Grapes");
        a1.add("Apple");
        a1.add("Chickoo");
        for (Object o : a1) {
            String s = (String)o;

```

```

        char first = s.charAt(0);
        char last = s.charAt(s.length()-1);
        if (first == 'o' || last == 'o' || first == 'O' || last == 'O') {
            System.out.println(s);
        }
    }
}

```

Output:

Potato
Orange
Tomato
Chickoo

Linked List:

- *LinkedList is a Concrete implementing class of **List Interface**.*
- *LinkedList class uses a doubly linked list to store the elements. It provides a linked-list structure. It inherits the Abstract List class and implement **List & Deque Interfaces**.*

Doubly Linked List:

- *Each Element in LinkedList is Object itself.*

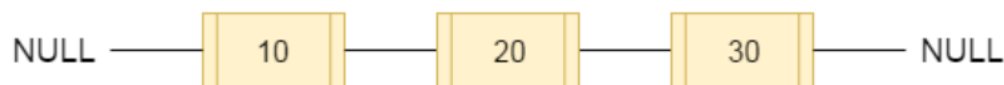


fig- doubly linked list

Constructors of LinkedList:

3. LinkedList ():

- *Creates an Empty LinkedList.*

4. LinkedList (Collection c):

- *Creates a list containing the elements of the specified collection, in the order they returned by the collection iterator.*

Program:

Insert the Node at the Beginning:

```

package Collection.linkedList;
class Node {
    int data;
    Node next;
    Node(int data) {
        this.data = data;
        next = null;
    }
}

```

```

public class InsertNode {
    public static Node insertAtFront(Node head, int new_data) {
        Node new_node = new Node(new_data);
        new_node.next = head;
        return new_node;
    }

    public static void printList(Node head) {
        Node curr = head;
        while (curr != null) {
            System.out.print(" " + curr.data);
            curr = curr.next;
        }
        System.out.println();
    }

    public static void main (String [] args) {
        Node head = new Node(2);
        head.next = new Node(3);
        head.next.next = new Node(4);
        head.next.next.next = new Node(5);
        System.out.println("Original Linked List:");
        printList(head);
        System.out.println("After inserting Nodes at the front:");
        int data = 1;
        head = insertAtFront(head, data);
        printList(head);
    }
}

```

Output:

Original Linked List:

2 3 4 5

After inserting Nodes at the front:

1 2 3 4 5

Delete the Node from LinkedList:

```

package Collection.linkedList;

public class RemoveNode {
    public static void main (String [] args) {
        Node head = new Node(1);
        head.next = new Node(2);
    }
}

```

```
        head.next.next = new Node(3);
        head.next.next.next = new Node(4);
        head.next.next.next.next = new Node(5);

        System.out.println("Original list: ");
        printList(head);
        head = deleteHead(head);
        System.out.println("List after deleting the head: ");
        printList(head);
    }

    public static Node deleteHead(Node head) {
        if (head == null) {
            return null;
        }
        Node temp = head;
        head = head.next;
        temp = null;
        return head;
    }

    public static void printList(Node head) {
        while (head != null) {
            System.out.print(head.data + ", ");
            head = head.next;
        }
        System.out.println("null");
    }
}
```

Output:**Original list:****1, 2, 3, 4, 5, null****List after deleting the head:****2, 3, 4, 5, null**

Difference Between ArrayList & LinkedList:

Sr.No	ArrayList	LinkedList
1.	ArrayList internally uses a Dynamic Array to store the elements	LinkedList internally uses a double Linked List to store the elements.
2.	Manipulation with ArrayList is slow because it internally uses an Array. If any element is removed from the Array, all the other elements are shifted in memory	Manipulation with LinkedList is faster than ArrayList because it uses a doubly linked list, so no bit shifting is required in memory.
3.	An ArrayList class can act as a list only because it implements List only.	LinkedList class can act as a list and queue both because it implements List and Deque interfaces
4.	ArrayList is better for storing and accessing data.	LinkedList is better for manipulating data.
5.	The memory location for the elements of an ArrayList is contiguous.	The location for the elements of a linked list is not contiguous.
6.	Generally, when an ArrayList is initialized, a default capacity of 10 is assigned.	There is no case of default capacity in a LinkedList. In LinkedList, an empty list is created when a LinkedList is initialized.
7.	Precisely, ArrayList is a Resizable Array.	LinkedList implements the doubly linked list of the list interface

Difference Between ArrayList & Vector:

Sr.No	ArrayList	Vector
1.	ArrayList is not Synchronized	Vector is Synchronized
2.	ArrayList increments 50% of the current Array Size if the number of elements exceeds its capacity	Vector increments 100% means doubles the Array Size of the total number of elements exceeds its capacity.
3.	ArrayList is not a Legacy Class. It is introduced in JDK 1.2	Vector is Legacy Class. It is introduced before JDK 1.2
4.	ArrayList is fast because it is non – synchronized	Vector is slow because it is synchronized.
5.	ArrayList uses the Iterator Interface to traverse the elements	Vector can use Iterator Interface or Enumeration Interface to traverse the elements
6.	ArrayList performance is High	Vector performance is Low
7.	Multiple Threads is allowed	Only one Thread is Allowed.

Java Generics:

- We can classify Collection into 2 categories –

- i. Non-Generic Collection.
- ii. Generic Collection

I. Non-Generic Collection:

- It is a Homogenous Collection of Elements.
- Every Element getting stored in this Collection is converted (Upcasted) to *java.lang.Object* type.

II. Generic Collection:

- It is a Homogeneous Collection of elements.
- Every element getting stored is not converted (Upcasted) to *java.lang.Object* type instead they are stored or converted into its own respective generic type.
- The return type of elements from Generic Collection will be same as the given Generic type.

Syntax for Generic Collection:

CollectionType<non-primitive datatype> variable = new CollectionType<non-primitive datatype>();

Example:

List<String> a = new ArrayList<String>(); //Upcasted ArrayList to List Interface.

Program:

```
package Collection.arrayList;
import java.util.List;
public class HomogeneousArrayList {
    public static void main (String[] args) {
        List<String> a1 = new ArrayList<String> ();
        a1.add ("Hello");
        a1.add (10); // CTE
        a1.add (true); // CTE
        a1.add ("Bye");
    }
}
```

Output:

Hello
Bye

Iterator: [java.util.Iterator → Interface]

- Iterator is used to iterate over a Collection.
- Iterator Interface has two abstract methods which helps us to perform iteration.
 - i. `hasNext()` → Boolean Type
 - ii. `next()` → Element Type

1. hasNext():

- It checks whether there is element to be accessible in collection (cursor), if present it return "true" else it return "false".

2. next():

- It is used to access the element and move the cursor to the next element.
- The return type of next() method is Generic Type. (If it is non-primitive collection, it returns non-primitive else if it is object type it returns object type)

Program:

```
package Collection.arrayList;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
public class IteratorMethods {
    public static void main (String [] args) {
        List<Integer> a = new ArrayList();
        a.add (10);
        a.add (20);
        a.add (30);
        Iterator i = a.iterator();
        while(i.hasNext()) {
            System.out.println(i.next());
        }
    }
}
```

Output:

```
10
20
30
```

Limitations of an Iterator: [Dis-Advantages]

- We can iterate only once.
- We cannot access the elements in reverse order.
- We cannot remove an element from collection during iteration. We get **ConcurrentModificationException**.

NoSuchElementException:

- In an Interface, when we try to access an element using `next()` method and if there is no element present, then we get **NoSuchElementException**.

List Iterator:

- List Iterator is the Sub-Interface of Iterator, it is defined in `java.util` Package.

Methods of ListIterator:**1. hasNext ():**

- To check whether the next element to accessed is present or not.

2. next ():

- It returns the current element and moves the cursor forward.

3. hasPrevious():

- It checks whether the previous element pointed by cursor is present or not.

4. previous ():

- It returns the element pointed by the cursor & moves the cursor backward.

5. remove ():

- It removes the current object pointed from the collection which is under iteration.

6. add (Object):

- It is used to add new object into the collection under iteration.

7. Set (Object):

- Replaces the element pointed by the cursor with the new element.

Program:

```
package Collection.arrayList;
import java.util.ArrayList;
import java.util.ListIterator;
public class ListIteratorExample {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add(10);
        a1.add(20);
        a1.add(30);
        a1.add(40);
        a1.add(50);
```



```

        ListIterator i = a1.listIterator();
        System.out.println(i.next());
        System.out.println(i.next());
        System.out.println(i.previous());
    }
}

```

Output:

```

10
20
20

```

Display Elements which Start and Ends with letter 'O':

```

package Collection.arrayList;
import java.util.ArrayList;
import java.util.ListIterator;
public class IteratorRemoveExample {
    public static void main (String [] args) {
        ArrayList a1 = new ArrayList();
        a1.add("Apple");
        a1.add("Mango");
        a1.add("Banana");
        a1.add("Grapes");
        a1.add("Chickoo");
        ListIterator i = a1.listIterator();
        while(i.hasNext()) {
            String s = (String) i.next();
            if(s.startsWith("o") || s.startsWith("O") || s.endsWith("o") || s.endsWith("O")) {
                System.out.println(s);
            }
        }
    }
}

```

Output:

```

Mango
Chickoo

```

Collection Class:

- *Collection Class is present in java.util Package.*
- *In Collection class we have many built-in static methods which helps us to perform utility actions such as –*
 - *Searching, Sorting, Shuffling, Max, Min, Rotate, Frequency, Swap, Reverse, Replace, Fill.*

Collection.sort():

- *Sort is a static method derived in Collection Class which internally uses **Comparable Interface** for its natural ordering of elements.*
- *It sorts the specified list into **ascending order**.*
- *It performs sorting for built-in objects.*
 - *Example: Integer, Character, Byte, etc [Wrapper Classes]*
- *If we want a sorting for customized object we need to implement the Comparable Interface & Override the compareTo() methods.*

Example:

- *A java Program for sorting Pre-defined objects.*

```
package Collection.arrayList;
import java.util.ArrayList;
import java.util.Collections;
public class SortingExample {
    public static void main (String [] args) {
        ArrayList<Integer> list = new ArrayList<Integer>();
        list.add(10);
        list.add(30);
        list.add(20);
        list.add(40);
        list.add(50);

        Collections.sort(list);

        for (Integer integer : list) {
            System.out.println(integer);
        }
    }
}
```

Output:

```
10
20
30
40
50
```

Comparable Interface:

- It is used by list type of collection for Natural sorting as well as Customized Sorting.
- We can use this Interface for custom object sorting.
- Comparable Interface is an example of Functional Interface, which contains only one abstract method. That is: `public int compareTo(T);`
- This Interface is derived in `java.lang` Package.

compareTo(T):

- This method compares the attributes and returns an int value based on –
 - If the first element is smaller than second element, method returns (-1).
 - If the first element is bigger than second element, method returns (1).
 - If the first element is equal to second element, method returns (0).

Sort the User-Defined Non-Primitive Objects using Student id: [Override compareTo() method]

```
package Comparable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
class Student implements
Comparable<Student> {
    int sid;
    String sname;
    public Student(int sid, String
sname) {
        this.sid = sid;
        this.sname = sname;
    }
    public String toString() {
        return "Student [sid = " +
sid + ", sname = " + sname + "]\n";
    }
    public int compareTo(Student s) {
        if (this.sid < s.sid)
            return -1;
        else if(this.sid > s.sid)
            return 1;
        return 0;
    }
}
public class StudentDriver {
    public static void main (String []
args) {
        ArrayList<Student> a1 =
new ArrayList<Student>();
```

Sort the User-Defined Non-Primitive Objects using Student Name: [Override compareTo() method]

```
package Comparable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
class Student implements
Comparable<Student>{
    int sid;
    String sname;
    public Student(){
    }
    public Student(int sid, String
sname) {
        this.sid = sid;
        this.sname = sname;
    }
    public String toString() {
        return "Student [sid = "
+ sid + ", sname = " + sname +
"]\n";
    }
    public int compareTo(Student
s) {
        return
this.sname.compareTo(s.sname);
    }
}
public class StudentDriver {
    public static void main (String
[] args) {
```

```
a1.add(new Student(1,"Vinay"));
a1.add(new Student(2,"Vijay"));
a1.add(new Student(4,"Suhas"));
a1.add(new Student(3,"Sada"));
Collections.sort(a1);
    for (Student student : a1) {
        System.out.println(student);
    }
}
```

Output:

```
Student [sid = 1, sname = Vinay]
Student [sid = 2, sname = Vijay]
Student [sid = 3, sname = Sada]
Student [sid = 4, sname = Suhas]
```

```
ArrayList<Student> a1 = new
ArrayList<Student>();
a1.add(new
Student(1,"Vinay"));
a1.add(new
Student(2,"Ajay"));
a1.add(new
Student(4,"Suhas"));
a1.add(new
Student(3,"Sada"));
Collections.sort(a1);
    for (Student student :
a1) {
        System.out.println(student);
    }
}
```

Output:

```
Student [sid = 2,
sname = Ajay]
Student [sid = 3,
sname = Sada]
Student [sid = 4,
sname = Suhas]
Student [sid = 1,
sname = Vinay]
```

Set Interface:

- Set is an Interface which is present in java.util package.
- It is a sub – interface of collection interface, therefore all the methods of collection are inherited in set interface.

Characteristics of Set:

- It is an unordered collection of elements (order of insertion is not maintained).
- Set does not allow duplicate elements.
- Set does not have indexing therefore we cannot access, insert or remove elements based on index.
- We can access the elements using –
 - i. Iterator
 - ii. For Each Loop

HashSet:

- It is a concrete Implementing class of Set Interface.
- It has all the methods implemented which are inherited from collection interface.

Constructor:**1. HashSet ():**

- Creates an empty HashSet Object of capacity 16.
- Load Factor: 3.7

2. HashSet (Collection C):

- Creates an HashSet Object of size 16 and copies elements from given Collection.

Program:

```
package Collection.set;
import java.util.HashSet;
public class HashSetExample {
    public static void main (String [] args) {
        HashSet<Integer> h = new HashSet<Integer>();
        h.add(10);
        h.add(1);
        h.add(21);
        h.add(35);
        h.add(54);
        h.add(5);
        h.add(60);
        h.add(744);
        h.add(805);
        h.add(9);
        System.out.println(h);
    }
}
```

Output:

[1, 35, 21, 5, 805, 54, 744, 9, 10, 60]

Program to Convert HashSet to ArrayList and Vica Versa:

```

package Collection.set;
import java.util.ArrayList;
import java.util.HashSet;

public class ConversionListAndSet {
    public static void main (String [] args) {
        ArrayList<String> names = new ArrayList<String> ();
        names.add("Vinay");
        names.add("Vijay");
        names.add("Suhas");
        names.add("Sadanand");
        names.add("Akash");
        System.out.println(names);

        HashSet<String> h = new HashSet<String>(names);
        System.out.println(h);

        ArrayList<String> a = new ArrayList<String>(h);
        System.out.println(a);
    }
}

```

Output:

```

[Vinay, Vijay, Suhas, Sadanand, Akash]
[Suhas, Vijay, Sadanand, Akash, Vinay]
[Suhas, Vijay, Sadanand, Akash, Vinay]

```

TreeSet:

- It is a concrete Implementing class of Set Interface.
- It has all the methods implemented which are inherited from collection interface, Set Interface, SortedSet Interface, NavigableSet Interface.

Constructors:**1. TreeSet ():**

- Constructs a new, empty tree set, sorted according to the natural ordering of its elements.

2. TreeSet (Collection c):

- Constructs a new tree set containing the elements in the specified collection, sported according to the natural ordering of its elements.

NOTE:

- TreeSet contains unique elements like HashSet.
- TreeSet doesn't allows Null Elements.
- TreeSet is non synchronized.

- *TreeSet maintains Ascending Order.*

Implementation:

```
import java.util.TreeSet;
```

```
public class TreeSetCreation {
    public static void main(String args[]) {
        // Create a TreeSet of Strings
        TreeSet<String> treeSet = new TreeSet<>();

        // Displaying the TreeSet (which is empty at this point)
        System.out.println("TreeSet elements: " + treeSet);
    }
}
```

Output:

TreeSet elements: []

Difference Between TreeSet & SortedSet:

Sr.No	Basic	TreeSet	SortedSet
1.	Classification	Concrete Class of Navigable Sub – Interface	Sub – Interface of Set Interface
2.	Insertion Order	TreeSet maintains Objects in Sorted Order	SortedSet maintains Object in Sorted Order.
3.	Initialization	TreeSet<Datatype> obj = new TreeSet<>();	It cannot be instantiated as it is a Sub – Interface.
4.	Methods	Contains more methods than SortedSet	Contains fewer methods than TreeSet.
5.	Example	Contains all methods of SortedSet. In Addition methods like ceiling(), floor(), higher(), lower(), etc	Contains methods like add(), addAll(), iterator(), retainAll(), etc

Difference Between HashSet & LinkedHashSet & TreeSet:

Sr.No	Features	HashSet	LinkedHashSet	TreeSet
1.	Internal Working	HashSet internally uses HashMap for Storing Objects	LinkedHashSet uses LinkedHashMap internally to store Objects	TreeSet uses TreeMap internally to store Objects
2.	When to Use?	If we don't want to maintain insertion Order but want to store unique Objects	If we want to maintain the insertion order of elements then use LinkedHashSet	If we want to sort the elements according to some Comparator then use TreeSet
3.	Order	HashSet does not maintain insertion Order	LinkedHashSet maintains the insertion order of Objects	Objects will be placed according to their natural ascending Order.
4.	Null Elements	HashSet allows only one null value	LinkedHashSet allows only one null value.	TreeSet does not permit null value. If we insert null value, it will throw NullPointerException
5.	Complexity of Operations	HashSet gives O(1) Complexity for insertion, removing and retrieving Objects.	LinkedHashSet gives O(1) Complexity for insertion, removing and retrieving Objects.	TreeSet gives O(log(n)) Complexity for insertion, removing and retrieving Objects.
6.	Compare	HashSet uses equals() and hashCode() methods to Compare Objects	LinkedHashSet uses equals() and hashCode() methods to Compare Objects	TreeSet uses compare() and compareTo() methods to Compare the Objects
7.	Performance	HashSet is better than LinkedHashSet & TreeSet	LinkedHashSet is slower than TreeSet. Almost similar to HashSet but slower because LinkedHashSet internally maintains LinkedList to maintain the insertion order of elements.	TreeSet performance is better than LinkedHashSet except for insertions and removal operations because it has to sort the elements after each insertion and removal operation.
8.	Syntax	HashSet obj = new HashSet ();	LinkedHashSet obj = new LinkedHashSet();	TreeSet obj = new TreeSet();

Difference Between HashSet & HashMap:

Sr.No	Basic	HashSet	HashMap
1.	Implements	<i>Set Interface</i>	<i>Map Interface</i>
2.	Duplicates	<i>NO</i>	<i>YES, duplicate values allowed but NO duplicate Keys allowed</i>
3.	Dummy Values	<i>YES</i>	<i>NO</i>
4.	Objects Required during add Operation	<i>1</i>	<i>2</i>
5.	Adding & Sorting Mechanism	<i>HashMap Object</i>	<i>Hashing Technique</i>
6.	Speed	<i>Slower than HashMap</i>	<i>Faster than HashSet because of Hashing Technique</i>
7.	Null	<i>Single Null Value</i>	<i>Single Null Key & any number of Null Values</i>
8.	Insertion Method	<i>Only one value is required for insertion process. add() methods is used for insertion</i>	<i>Two values are required for insertion process. put() method is used for insertion</i>
9.	Data Storage	<i>Data is stored as Objects</i>	<i>Data is stored as Key-Value Pair</i>
10.	Complexity	<i>O(n)</i>	<i>O(1)</i>

Queue Interface:

- Queue is an Interface which is present in java.util package.
- It is a sub – interface of collection interface, therefore all the methods of collection are inherited in Queue interface.

Characteristics of Queue:

- Queue keep the elements in FIFO (First In First Out) Order.
- Queue is an Ordered List of Objects.
- Insertion of elements occurs at end of the list and removal of elements occur at the beginning of the list.
- Queues present in java.util package is known as Unbounded Queues.
-
- Queue Provides additional methods for insertion, extraction and inspection operations.

	Throws Exception	Returns Special Value
Insert	add (e)	offer (e)
Remove	remove ()	poll ()
Examine	element ()	peek ()

PriorityQueue:

- It is a Concrete Implementing class of Queue Interface.
- It has all the methods implemented which are inherited from collection interface & Queue Interface.
- Sometimes the elements of the Queue needed to processed according to the priority, that's where a PriorityQueue comes into action.

Constructors:**1. PriorityQueue ():**

- Creates a PriorityQueue with default initial Capacity (**11**) that Orders its elements according to their **natural ordering**.

2. PriorityQueue (Collection c):

- Creates a PriorityQueue containing the elements in the specified collection.

3. PriorityQueue(int initialCapacity):

- Creates a PriorityQueue with the specified initial capacity that orders its elements according to their **natural ordering**.

Implementation:

```
import java.util.*;
```

```
class PriorityQueueDemo {
    public static void main(String args[]) {
        PriorityQueue<Integer> pQueue = new PriorityQueue<Integer>();
        pQueue.add(10);
    }
}
```

```

        pQueue.add(20);
        pQueue.add(15);
        System.out.println(pQueue.peek());
        System.out.println(pQueue.poll());
        System.out.println(pQueue.peek());
    }
}

```

Output:

10

10

15

Deque Interface:

- Deque Interface is present in java.util package.
- Deque is known as double ended queue.
- It is a sub – type of Queue Interface.
- Deque Interface supports addition as well as removal of elements **from both ends** of the Data Structure.
- Deque can be used as **Stack or Queue**.
- Deque also supports LIFO (Last In First Out) operation.
- We can access the elements using –
 - i. Iterator
 - ii. For Each Loop

NOTE:

- Deque provides additional methods for insert, remove, examine the element from both ends.

	First Element (Head)		Last Element (Tail)	
	Throws Exception	Special Value	Throws Exception	Special Value
Insert	<i>addFirst (e)</i>	<i>offerFirst (e)</i>	<i>addLast (e)</i>	<i>offerLast (e)</i>
Remove	<i>removeFirst ()</i>	<i>pollFirst ()</i>	<i>removeLast ()</i>	<i>pollLast ()</i>
Examine	<i>getFirst ()</i>	<i>peekFirst ()</i>	<i>getLast ()</i>	<i>peekLast ()</i>

ArrayDeque:

- It is a Concrete Implementing class of Queue Interface.
- It has all the methods implemented which are inherited from collection interface, Queue Interface & Deque Interface.
- It is a Resizable Array implementation of the Deque Interface.

Constructors:

1. ArrayDeque ():

- Constructs an empty Array Deque with initial Capacity sufficient to hold **16** elements.

2. ArrayDeque (Collection c):

- Constructs a deque containing the elements of the specified collection, in the order they are returned by the collection iterator.

Implementation:

```
import java.util.ArrayDeque;
import java.util.Deque;

public class Example {
    public static void main(String[] args) {
        Deque<Integer> deque = new ArrayDeque<>();
        deque.addFirst(1);
        deque.addLast(2);
        int first = deque.removeFirst();
        int last = deque.removeLast();
        System.out.println("First: " + first + ", Last: " + last);
    }
}
```

Output:

First: 1, Last: 2

NOTE:

- It is **recommended** by Java Documentation to use ArrayDeque to implement the **Stack** in Java.
- ArrayDeque is non synchronized class.

Difference Between Stack & Queue:

Sr.No	Feature	Stack	Queue
1.	Definition	A linear data structure that follows Last IN First Out (LIFO) Principle	A linear data structure that follows First In First Out (FIFO) Principle
2.	Primary Operations	1. Push (add an item) 2. pop (remove an item) 3. Peek (view the top item)	1. Enqueue (add an item) 2. dequeue (remove an item) 3. front (view the first item) 4. rear (view the last item)
3.	Insertion / Removal	Elements are added & removed from the same end (top)	Elements are added at the rear & removed from the front
4.	Use Cases	1. Function Call Management (call stack) 2. Expression evaluation and syntax parsing 3. Undo Mechanisms in text editors	1. Scheduling processes in OS 2. Managing Request in a Printer Queue 3. Breadth – First Search in Graphs
5.	Examples	1. Browser History (back button) 2. Reversing a word	1. Customer Service lines 2. CPU task Scheduling
6.	Real-World Analogy	A stack of plates: you add and remove plates from the top	A queue at a ticket counter: the first person in line is the first to be served
7.	Complexity (Amortized)	1. Push: O(1), 2. Pop: O(1), 3. Peek: O(1)	1. Enqueue: O(1), 2. Dequeue: O(1), 3. Front: O(1), 4. Rear: O(1)
8.	Memory Structure	Typically uses a contiguous block of memory or LinkedList	Typically uses a Circular Buffer or LinkedList
9.	Implementation	Can be Implemented using Arrays or LinkedList	Can be implemented using Arrays, LinkedList or Circular Buffers.

- **Vector is a Legacy Class from List Interface.**
- **Stack is a Legacy Class from List Interface which extends Vector Class.**
- **HashTable is Legacy Class from Map Hierarchy, it is similar to HashMap. HashTable is Synchronized Class.**